

RTL Design Sherpa

ALU-INFER Micro-Architecture Specification 0.50

October 18, 2026

Table of Contents

1 Alu Infer Mas Index....12

2 ALU-INFER — Cell Inference ALU Product Requirements Document....13

2.1 1. Identity and scope....14

2.2 2. Element model....15

2.3 3. Function set (v1)....17

2.3.1 Function-set roadmap (owner ruling 2026-10-19)....18

2.4 4. Interface contract....20

2.5 5. Control interface — one RV32I core per tetrad, in-band everything....24

2.6 6. Verification hooks....26

2.7 7. Deferred and growth....27

2.8 8. Open questions....28

2.9 9. Dependencies....29

2.10 10. References....30

3 Cell — Compute Engine IP Product Requirements Document....32

3.1 1. Identity and scope....33

3.2 2. Architecture....34

3.2.1 Figure 1: Cell top-level block plan (house style — three flows: programming down the center, operand fill on the left, results and credits back up the right)....34

3.3 3. Dataflow....37

3.4 4. ALU and execution FSM....38

3.5 5. Packet interface....39

3.6 6. Credit behavior....40

3.6.1 6.1 Boot advertisement....40
3.6.2 6.2 Consumed-line credit returns....40
3.6.3 6.3 Straps on the result path....40
3.7 7. Config registers....42
3.8 8. Status and observability....45
3.9 9. Verification hooks....46
3.10 10. Dependencies....47
3.11 11. Deferred....48
3.12 12. Open questions....49
3.13 References....50
4 alu-infer — Cell Inference ALU (definition)....51
5 Tile — Compute Engine Tile Subsystem Specification....52
5.1 1. Scope....53
5.2 2. Hierarchy....54
5.3 3. Connectivity....55
5.4 4. Modules....56
5.5 5. Observation (MonBus aggregation)....57
5.6 6. Interfaces....58
5.7 7. Modes....59
5.8 8. Rapids attachment....60
5.9 9. Clocking and reset....61
5.10 10. Open items....62
5.11 11. Hierarchical tile, root nodes, addressing (ADOPTED 2026-10-16)....64
5.11.1 11.1 Hierarchy....64

5.11.2 11.2 Payload types....64
5.11.3 11.3 Destination coordinate....65
5.11.4 11.4 Root failover....66
5.11.5 11.5 Packet header....67
5.11.6 11.6 Transport....68
5.11.7 11.7 Where the fields come from....69
5.11.8 11.8 Operations....69
5.11.9 11.9 End-to-end credits....71
5.11.10 11.10 Credit advertisement....73
5.11.11 11.11 Cell-issued descriptors....74
5.11.12 11.12 RAPIDS implementation status (2026-10-17)....75
5.11.13 11.13 Tetrad control core (owner direction 2026-10-17; deferred 2026-10-18; ruling 2026-10-19)....77
5.11.14 11.14 Result flush to the tile I/O (owner direction 2026-10-18; defined 2026-10-19)....78
5.11.15 11.15 Multi-path routing and failover: open hazards (2026-10-18)....80
5.12 12. Milestones....81
5.13 References....82
6 KESTREL-RV32I Micro-Architecture Specification Index....83
6.1 Document Organization....84
6.1.1 Front Matter....84
6.1.2 Chapter 1: Overview....84
6.1.3 Chapter 2: Functional Blocks....84
6.1.4 Chapter 3: Internal Interfaces....84

6.1.5 Chapter 4: Micro-Architecture Behavior....84
6.1.6 Chapter 5: Verification Hooks....84
6.1.7 Chapter 6: Configuration....84
6.2 Related Documentation....85
7 rtl-math module index....86
7.1 Overview....87
7.2 Module categories....88
7.2.1 Basic arithmetic....88
7.2.2 Advanced adders....88
7.2.3 Division....89
7.2.4 Subtraction....89
7.2.5 Multiplication....89
7.2.6 BF16 floating-point arithmetic....90
7.2.7 FP16 (half-precision) floating-point....91
7.2.8 FP32 (single-precision) floating-point....91
7.2.9 FP8 (8-bit floating-point)....91
7.2.10 IEEE 754-2008 compliant arithmetic....91
7.2.11 Compressors and prefix cells....91
7.3 Related Modules....93
7.4 Navigation....94
8 Document Information....95
8.1 Overview....95
8.1.1 ALU-INFER Micro-Architecture Specification....95
8.1.2 Document Purpose....96

8.1.3 Intended Audience....96
8.2 Design Notes....97
8.2.1 Revision History....97
8.3 References....102
8.3.1 Related Documents....102
9 Design Philosophy and Datapath Organization....103
9.1 What This Block Is....103
9.2 The Dataflow in One Paragraph....104
9.2.1 Figure 1.1: ALU-INFER datapath overview (definition level)....104
9.3 Relationship to the Cell and the Tetrad Core....106
9.4 Status of This Specification....107
10 alu_infer_top (module tree)....108
10.1 Purpose....108
10.2 Module Tree....109
10.2.1 Figure 2.1: ALU-INFER full block diagram....110
10.3 Parameters....111
10.4 Clock and Reset....112
11 The Seven Function Rows....113
11.1 Row Summary....114
11.2 Per-Row Notes....116
11.3 What Every Row Shares....117
12 The FP32 MAC Accumulator....118
12.1 Why fp32....118
12.2 Structure....119

12.3 The Loop-Carried Dependence (why the row is split)....120
12.4 Presentation Semantics (MAS decision M7)....121
12.5 Isolation Invariants....122
13 Result Select and Line Composer....123
13.1 Result Select (alu_infer_select)....123
13.2 Line Composer (alu_infer_compose)....124
13.3 Registered Config....125
14 The ALU↔FSM Interface....126
14.1 Signal Contract....127
14.2 Handshake Rules....130
14.2.1 Waveform 3.1: Elastic handshake — natural latency, output stall, ready stays high....131
14.2.2 Waveform 3.2: Back-to-back issue — mixed latencies, in-order results....132
14.3 Boundary Notes....133
15 The Compute Cycle, per Opcode Class....134
15.1 Unary Rows (copy, relu)....135
15.2 Binary Rows (add, sub, mul, min/max)....136
15.3 Compare / Threshold....137
15.4 MAC (the chain that matters)....138
15.4.1 Waveform 4.1: MAC chain — clear, three accumulates, drain....138
15.5 Overlapping Invocations....140
16 Verification Hooks and Observability....141
16.1 The Golden Model....142

16.2 Built-in Cross-check....143
16.3 Coverage Matrix....144
16.4 Elastic and Tag-Join Behavior....145
16.5 Observability....146
16.6 Test Levels....147
17 Parameters, Encodings, and Open Items....148
17.1 Parameters....148
17.2 func_sel Encoding....149
17.3 acc_ctrl Encoding....150
17.4 Filelists....151
17.5 Open Items Register....152

List of Figures

Figure 1: Cell top-level block plan (house style — three flows: programming down the center, operand fill on the left, results and credits back up the right).....33
Figure 1.1: ALU-INFER datapath overview (definition level).....103
Figure 2.1: ALU-INFER full block diagram.....109

List of Tables

Table 1: MonBus sources per level....56
Table 2: Proposed destination coordinate, {ext, level, X, Y, Z}, 15 bits at COORD_W = 4....64
Table 3: Proposed tile packet header fields (beyond delta's existing ones)....66
Table 4: Proposed header packing....67
Table 5: Proposed operations between RAPIDS and the tile....68
Table 6: Credits toward RAPIDS....71
Table 7: Credit packet operations....72
Table 8: Document information....94
Table 9: Revision history....96
Table 10: Table 2.1: alu_infer_top parameters....110
Table 11: Table 2.2: the v1 function rows (normative list in PRD §3)....113
Table 12: Table 2.3: accumulator actions....118
Table 13: Table 3.1: ALU↔FSM signals....126
Table 14: Table 5.1: unit coverage matrix....143
Table 15: Table 6.1: func_sel fields....148
Table 16: Table 6.2: acc_ctrl encoding (accepted 2026-10-17 — "works for now"; revisit at FPGA bring-up if the mac row is reworked)....149
Table 17: Table 6.3: open items M1–M8....151

List of Waveforms

Waveform 3.1: Elastic handshake — natural latency, output stall, ready stays high 130
Waveform 3.2: Back-to-back issue — mixed latencies, in-order results.....131
Waveform 4.1: MAC chain — clear, three accumulates, drain.....137

1 Alu Infer Mas Index

Generated: 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

← Back to compute-eng-ip

2 ALU-INFER — Cell Inference ALU Product Requirements Document

Version: 0.50 (2026-10-18 — **revs pinned at 0.50 per owner, "for now,"** while the ALU heads to the board; this rev accumulates every ruling to date). Accumulated since the v0.3 base: **tag-based elastic alignment** (§4 invariant 3) — each loaded invocation takes an 8-bit tag, common to all ops that must execute in parallel; a FIFO tracks tags in flight; an op asserts its valid only when every op of its tag group is ready; tags stay inside the block and the seam keeps its two valid/ready pairs only; enables the §7 growth item (deep-pipelining the multiplier / FMA rows with the adder's `PIPE_STAGE_*` pattern). **Tetrad-engine deferral / gatling ruling 2026-10-19** (§5) — the per-tetrad core is defined after the cell is fully defined and working; the owner-authored gatling-rv32itetrad barrel-core PRD is the **selected sequencer** for its tetrad (kestrel-class stays a `CORE_FLAVOR` candidate). **Port-width correction** — operand ports stay `LINE_W` each (one line per side, both captured in the same clock); the earlier "widening" was a line-vs-lane mis-count. **Owner accept-and-defer** — M7 and `acc_ctrl` accepted as drafted; per-row latency work deferred to FPGA bring-up (M8). **Elastic-pipeline interface** — valid/ready at both ends, backpressure freezes the alignment tracking, II=1 while draining, no padded stages on quick rows, in-order results. **Base decisions** — bf16 element model on the rtl/math `math_bf16_*` library, fp32 MAC accumulate, one RV32I core per tetrad with in-band everything. **Status:** Pre-HAS PRD, and the **normative definition of the cell's ALU block**: the RTL implementation lives in this component (`alu-infer/rtl/`, unit tests in `alu-infer/dv/tests/`; moved from `cell/rtl/` 2026-10-20) and the **cell** component integrates it. Where this document and the cell PRD touch the ALU, this one governs (the cell PRD was reconciled to it 2026-10-17; see §2).

2.1 1. Identity and scope

alu-infer is the compute engine of the **cell**: the function array the cell's execution FSM drives to turn input-memory lines into result-memory lines. It is an *inference* ALU — the arithmetic a neural-network layer needs (add, multiply, fused accumulate, min/max, threshold, ReLU-family activations), not a general-purpose ISA datapath.

Component base (owner direction 2026-10-17): the implementation is built mostly from the `math_bf16_*` family of `rtl/math/` — the repo's existing brain-float arithmetic library. Nearly every v1 function row has a block there (adder, multiplier, FMA, comparator, min/max, ReLU, and the rest of the activation set), so the ALU is the math library's inference consumer the same way the falcon suite's garuda rung is its IEEE-fp32 consumer. Rows without a bf16 block (copy, and the compare *mask* packing) are wires and glue.

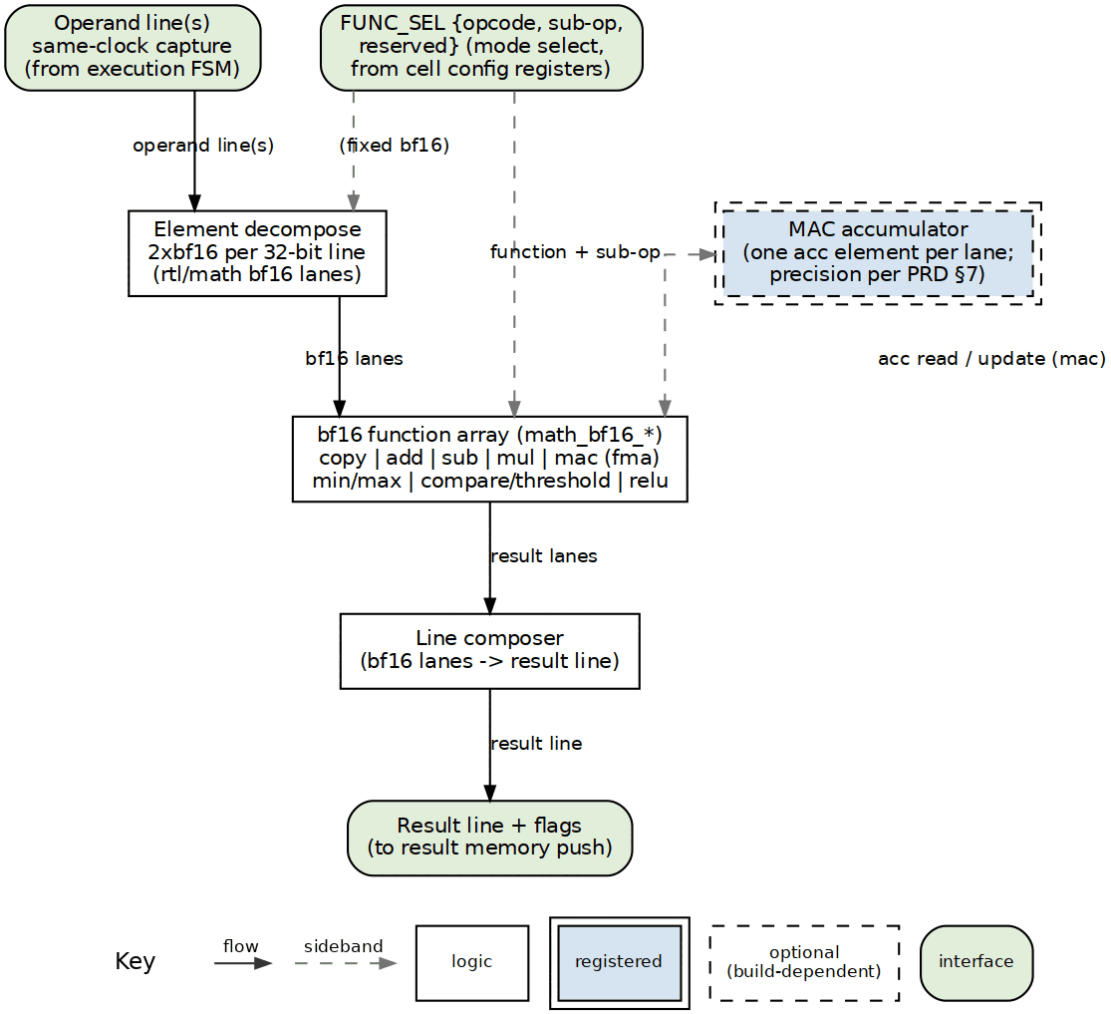
Placement (owner decision 2026-10-17, updated 2026-10-20): this directory holds the product definition, verification doctrine, the RTL (`rtl/`, SystemVerilog), and its cocotb unit tests (`dv/tests/`) — organized like the repo's other component areas (the dma-ip/stream pattern: flat RTL with per-block filelists and a lint master, a four-line area Makefile, and a discoverable `dv/tests` suite). The `cell/` component integrates the block as the cell's "ALU + execution FSM" element: the cell filelists include this block's filelist, and cell-level tests will verify it with the memories, packetizer, and credit logic it serves. The block boundaries in the cell PRD were chosen so a future CGRA array replaces this ALU without disturbing the network, credit, or observation contracts (cell PRD §11); this PRD's interface contract (§4) is written to that same seam.

Out of scope: the execution FSM's sequencing policy, the memories, the packetizer, credit/grant logic, and the delta node (all cell PRD territory).

2.2 2. Element model

Owner direction 2026-10-17: a **32-bit line is read as 2×bf16 elements** — two 16-bit brain-float lanes — and the function array is built from `math_bf16_*` blocks. bf16 keeps fp32's 8-bit exponent and truncates the mantissa to 7 bits, which is why the library's inference blocks (and the quantization literature behind them) chose it: full fp32 range at half the storage. The packed-line structure of cell PRD §4 is retained: the configured function applies to every element in the line **in parallel** (2 lanes per 32-bit line), and wider lines (cell PRD §12) scale the lane count, never the element width.

Supersession, reconciled. Cell PRD §4's element-width note (owner, 2026-10-16) defined B4/B8/B16/B32 *integer* widths; that note is superseded by this bf16 model. The cell PRD was updated to match on 2026-10-17 — the B4–B32 integer model is not planned (a companion integer mode is deferred, §7). One source per fact: this PRD owns the ALU's element model; the cell PRD links here rather than restating it.



ALU-INFER datapath

2.3 3. Function set (v1)

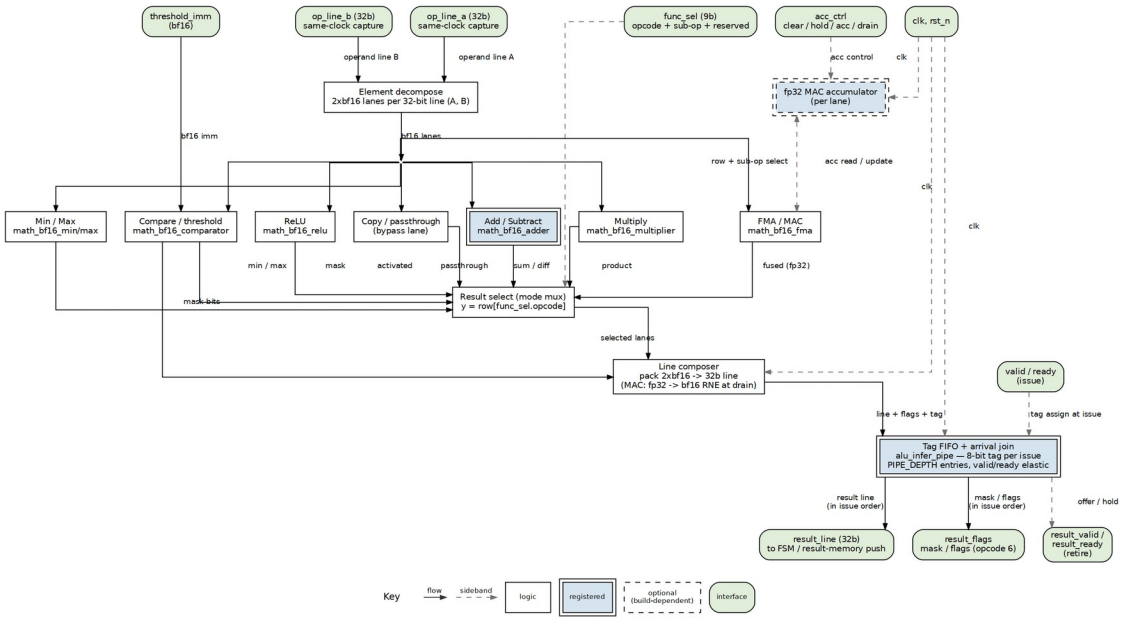
Opcode rows on the bf16 base; the `math_bf16_*` block named in each row is the implementation base in the ALU's RTL. Each row is an RTL table entry, not a new block.

Opcode	Function	`rtl/math` base	Operand pattern
0	copy / passthrough	wires	1 line → 1 line
1	add	`math_bf16_adder`	2 lines → 1 line
2	sub	`math_bf16_adder` (sign-flipped operand)	2 lines → 1 line
3	mul	`math_bf16_multiplier` (`math_bf16_mantissa_mult` + `math_bf16_exponent_adder`)	2 lines → 1 line
4	mac	`math_bf16_fma` — fused, fp32 accumulate (§3 note)	2 lines + accumulator → 1 line
5	min / max (sub-op 1b)	`math_bf16_min` / `math_bf16_max` (via `math_bf16_comparator`)	2 lines → 1 line
6	compare / threshold (sub-op)	`math_bf16_comparator`	2 lines or line + imm → 1 mask line
7	relu	`math_bf16_relu`	1 line → 1 line

MAC accumulation (decided 2026-10-17): the FMA row computes the bf16 product **exactly in fp32** (8×8 significand bits fit fp32's 24) and accumulates in a **per-lane fp32 accumulator**; one rounding to bf16 per result line at drain. This is the production bf16 pattern; it promotes the v0.2 open question to a decision.

Opcode space reserves 5 bits; sub-ops 2 bits; 8–31 are growth. The fused family (`math_bf16_fma`) makes the mac row the workhorse an inference workload expects.

The full block diagram — every v1 row with its `rtl/math` block, the fp32 accumulator, and the sideband plane:



ALU-INFER block diagram

Growth rows (same table, later opcodes — the library blocks already exist): leaky-relu / gelu / sigmoid / silu / tanh activations (`math_bf16_leaky_relu`, `math_bf16_gelu`, `math_bf16_sigmoid`, `math_bf16_silu`, `math_bf16_tanh`); reciprocal and divide (`math_bf16_reciprocal`, `math_bf16_newton_raphson_recip`, `math_bf16_goldschmidt_div`); exp2 / log2 scaling (`math_bf16_exp2`, `math_bf16_log2`, `math_bf16_log2_scale`); int8 drain (`math_bf16_scale_to_int8`); format conversions (`math_bf16_to_fp32`, `math_bf16_to_fp16`, `math_bf16_to_fp8_e4m3/e5m2`).

`FUNC_SEL` (cell PRD §7) carries {opcode 5b, sub-op 2b, element-width 2b} — the element-width field is **reserved** under this model (bf16 is the only v1 element width); its final disposition is §8.

2.3.1 Function-set roadmap (owner ruling 2026-10-19)

The v1 rows 0–7 in the table above stand unchanged. Row additions after FPGA bring-up, in priority order:

1. **reduce** — sum/max tree over a span of lines into the fp32 accumulator; unlocks softmax/layernorm/pooling.
2. **PWL/LUT activation** — 256-entry LUT + linear interpolation covering sigmoid/tanh/gelu/exp; exp plus reciprocal-multiply covers softmax's divide. No true divider is planned.
3. **const/bias** — emit the immediate register as a result line so programs can build bias lines without memory traffic.

Format roadmap, via the reserved `func_sel[8:7]` element-width field (open item M6): **int8** (4 lanes per 32-bit line) first — the same MAC fabric runs 4 MACs/clk and teaches quantization; **fp32** (1 lane per line) second — a general vector engine. **bf16 stays the v1 format.**

No-fork rationale: there are no A/B/C ALU variants — one ALU per cell; heterogeneity belongs at cell/tile granularity because delta/tile is payload-agnostic. The `alu-rv32i` and `alu-rv32f` blocks are ISA-datapath ALUs for the falcon cores (a different role, not variants of this one).

2.4 4. Interface contract

The contract the cell implementation obeys at the ALU↔FSM seam:

Signal group	Direction	Width	Notes
`op_line_a`, `op_line_b`	in	LINE_W each	operands — the function's `OP_LINES` count is across both ports (1 for copy/relu: the A line only; 2 for binary/mac: the A line and the B line); both lines are presented **in the same clock** and captured at the issue edge (owner ruling 2026-10-17: two ops scheduled in the same clock finish at the same time)
`threshold_imm`	in	16 (bf16)	immediate operand for opcode 6
`func_sel`	in	9b	{opcode, sub-op, reserved} — the legacy element-width field is reserved (§3)
`acc_ctrl`	in	—	accumulator clear / hold / accumulate (opcode 4 state)
`result_line`	out	LINE_W	one line per

Signal group	Direction	Width	Notes
			invocation for all v1 rows (2 bf16 lanes, fp32-rounded at drain for opcode 4)
`result_flags`	out	LINE_W	opcode-6 mask — one bit per lane, packed low (M5 closed 2026-10-17, owner ruling); zero for other rows
`valid` / `ready`	in handshake	1/1	elastic issue: the invocation enters the row pipeline when `valid && ready`; `ready` deasserts only under downstream backpressure, so a new invocation may be accepted **every clock** while the output drains (II=1); the FSM holds `valid` and its operands/config stable while `ready` is low
`result_valid` / `result_ready`	out handshake	1/1	elastic retire: each result is offered exactly `ROW_LATENCY` cycles after its

Signal group	Direction	Width	Notes
			issue (continuous drain) and held with stable data until accepted; results retire strictly in issue order

Invariants:

1. **Lane independence.** Except for opcode 4's accumulator, no lane of a result line depends on any other lane of the same or the other operand line. A lane can be checked against a scalar golden reference in isolation.
2. **Accumulator isolation.** The fp32 accumulator is per-lane, private to opcode 4, cleared only by `acc_ctrl`.
3. **Registered shell, uniform presentation, elastic timing.** Rows may be combinational or pipelined (cell MAS decides); the contract is a valid/ready elastic pipeline either way: every result is **presented** exactly `ROW_LATENCY` cycles after its issue, in issue order, when the output drains continuously. Quick rows compute in parallel at their natural speed and the block's alignment mechanism delays their presentation — no padded pipe stages on fast rows. **Tag-based alignment (owner ruling 2026-10-18):** when an invocation is loaded it takes an 8-bit tag, common to all ops that must execute in parallel; a FIFO tracks the tags in flight, and an op asserts its valid only when every op of its tag group is ready — the join yields group-aligned, in-order, elastic retirement without per-op timers. The v1-as-built block realizes the same contract with the alignment-timer mailbox of the MAS; the tag mechanism is the alignment architecture when rows gain uneven pipeline depths (§7). Backpressure propagates: a stalled consumer holds `result_valid` (data stable), freezes the alignment tracking, and deasserts `ready` at the input once the pipe fills; on release the pipeline resumes where it stopped. **Scheduler-paced feedback.** Whenever results feed back — opcode 4's chain today, any future recurrence — the scheduler owns the pace entirely: it shapes the `valid` stream and obeys `ready`, and the ALU makes no autonomous pacing decisions (owner ruling 2026-10-17). **No tags on this seam** — the tags above live inside the block; seam flow

control is the two valid/ready pairs alone, with no error signal. Steady state, un-stalled, is one invocation per clock regardless of depth.

4. **Function-table growth.** New opcodes and sub-ops are additive rows; they must not change §4's handshake or the lane-independence invariant.

2.5 5. Control interface — one RV32I core per tetrad, in-band everything

Owner ruling 2026-10-19: gatling sequences the four ALUs of its adjacent cells. The barrel RV32I core is the programmable sequencer for its tetrad — each context runs a small program whose POP presents operands to the cell's alu-infer issue handshake and whose PUSH retires a result line; the ALU keeps its elastic valid/ready seam and tag-alignment contract unchanged. Gatling is also a barrel RISC-V with custom commands (Xcell: CFG/POP/PUSH/ECTX/FLUSH/KICK/WAIT) — the program is the schedule. Gatling never originates config traffic; cell configuration arrives in-band over delta.

The tile blocks take **all data, config, and programming in-band on delta** (owner ruling 2026-10-17). The programming stream rides with the data as `cfg` packets (cell PRD §3); the cells' demux writes the §7 register map from the stream — **the stream is the programmer**. Consequences, decided with the owner:

- **The tetrad controller is deliberately not a config originator.** No config traffic is generated by the compute-side control core; configs arrive inline with the delta traffic. There is no per-tetrad config broadcaster to design or verify.
- **One RV32I core per tetrad** (4 cells), four per tile: the local programmable agent for its tetrad. The owner-authored `gatling-rv32itetrad` barrel RISC-V is the tetrad sequencer (owner ruling 2026-10-19). **Kestrel-class** — single-cycle, CPI 1, the repo's proven core (rv32ui battery and riscv-formal green) — remains a `CORE_FLAVOR` candidate via the same swap pattern HIVE-C uses. **Not SERV:** SERV is the repo's bit-serial *agent* core, and its role is the Ocelus observability infrastructure (hive-serv monitors) — its CPI class is wrong for compute control, and its job is already taken.
- **Duties** — local reaction, not programming: read cell status and IRQs (done, error, grant-starved, credit-timeout — cell PRD §8) over the local AXIL register plane; run small decision firmware (layer-boundary detection, flush policy — the cell PRD §12 open item; local event bookkeeping); exercise *operational* registers (flush-and-send, soft reset, IRQ clear — the AXIL-for-registers plane, not fabric config). Its own firmware arrives in-band like everything else's, booted through the repo's loader-window pattern.
- **Custom instructions stay minimal** (gatling Xcell subset in the custom-0 opcode space): KICK (funct3=100) starts its tetrad's cells once local conditions hold, WAIT (funct3=101) stalls until the tetrad's group-done bitmask is set; POP/PUSH present operands and retire results across the

ALU↔FSM seam. The gatling PRD v0.2 carries the exact encodings; the ALU's elastic handshake and tag-alignment contract are unchanged. The array's programming model remains the in-band stream; the core's firmware programs only its own reactions.

- **Why per-tetrad:** the tetrad is the tile's structural group; one group-done per four cells; four independent control points allow layer pipelining across tetrads without sixteen instruction streams or sixteen I-mems. SIMD width per tetrad: 4 cells × 2 bf16 lanes = 8 bf16 lanes stepping together; the 16-cell tile is 32 bf16 lanes.
- **Observability stays with Ocelus.** The tile's MonBus aggregation and the SERV-based monitor plane are untouched; the tetrad core reads registers and reacts — it is control, not instrumentation (§8 lists the boundary as an open detail).
- **Deferral (owner direction 2026-10-18):** the tetrad engine — core flavor, attach, and duties above — is defined **after the cell is fully defined and working**. The cell is the lowest module (cell PRD v0.3 §1); the owner-authored `gatling-rv32itetrad` barrel-core PRD (2026-10-17) is the selected sequencer, with kestrel-class remaining a candidate flavor via `CORE_FLAVOR`. The in-band-everything model in this section is unchanged by the deferral.

2.6 6. Verification hooks

- Stack per house convention: cocotb + rds-dv BFMs (cell PRD §9); the ALU's unit tests live in the cell's `dv/` tree since the RTL does.
- **Golden reference:** a small bf16 numerics model (fp32 computation per lane, rounded to bf16 RNE) — matching how the math library's own dv checks the bf16 blocks; lanes compared independently, which §4 invariant 1 makes a strong check. Opcode 4 checks the fp32-accumulate chain (long MAC runs, clear/hold corners, single-round-at-drain).
- **Special values are mandatory corners**, not extras: NaN payloads, ±inf, ±0, and denormals/FTZ behavior (the library's `SUBNORMAL_SUPPORT` parameter lineage — see the math references index) on every arithmetic row.
- **Activation rows** check against the library's reference approximations (piecewise/LUT), with the accuracy class stated per row in the cell MAS.
- **Cross-check against `rtl/math`:** the ALU's rows instantiate the library blocks; a unit-vs-library diff on identical operands is a bug in exactly one of them.
- **Controller bring-up** reuses the kestrel battery, loader contract, and lockstep flow unchanged (the HIVE-C precedent); the kick/wait extension and the in-band cfg stream are verified at the cell testbench level — the cfg path already exists in the cell PRD's dataflow tests.
- Coverage matrix: opcode (8) × special-value class × lane position (2 per line), plus accumulator chains and threshold immediate corners. The cell PRD's credit-loop tests already verify lines-in → credits-out independently of the datapath.

2.7 7. Deferred and growth

- **Deep-pipelined multiplier / FMA rows** — the combinational fp32 FMA (with its accumulator feedback) is the timing risk at FPGA bring-up; pipeline it with the adder's `PIPE_STAGE_*` pattern. Mixed row depths then stay II=1 with in-order retirement under the tag-based alignment ruling (§4 invariant 3), which "naturally takes care of the elastic nature" (owner 2026-10-18). A single lane's serial MAC dependency is arithmetic, not implementation — pipelining does not change it.
- **Activation expansion** (gelu, sigmoid, silu, tanh, leaky-relu) — library blocks exist; table rows once an owner priority calls for them.
- **Reciprocal / divide, exp2 / log2, int8 drain, format conversions** — library blocks exist (§3 growth list); same path.
- **fp8 element lanes** — `math_fp8_e4m3_*` / `math_fp8_e5m2_*` families exist; a mixed bf16/fp8 mode is an element-model extension, not v1.
- **Companion integer mode** (B4–B32) — deferred, and not currently planned; the bf16 model is the accelerator's single datapath (§2).
- **CGRA replacement** — per cell PRD §11, an array replaces this ALU behind the same FSM handshake.
- Context switching / multi-function pipelines and cell-initiated grant requests (both cell PRD §11) — cross-referenced, not restated.

2.8 8. Open questions

- **NaN / inf semantics in min/max and activations.** IEEE 754-2008 `minNum`/`maxNum` quiet-NaN propagation vs the inference-convention "ignore NaN" — the library's `math_bf16_min/max` and `math_bf16_comparator` semantics decide; state the choice in the cell MAS.
- **Rounding and exception policy.** RNE throughout per the library; a dynamic rounding mode is out of scope (garuda's RNE ruling is the precedent). Whether the ALU reports bf16 flag equivalents (the library's blocks carry IEEE exception outputs) into `result_flags`, or leaves them unobserved, is a cell-observability decision.
- **Denormal / FTZ behavior** on the arithmetic rows — follow each library block's `SUBNORMAL_SUPPORT` default, or force one policy cell-wide?
- **Tetrad-controller boundary details** — the core's program-load path and local memory map, the kick/wait encoding, and whether it ever takes a MonBus observation tap (default: no — Ocelus/SERV owns observability). All land in the cell/tile MAS, not this PRD.
- **Immediate encoding** for opcode 6 (`threshold_imm` as a raw bf16 bit pattern vs a small int compared after convert).
- **Result-flag line format** for opcode 6 — **closed 2026-10-17** (owner ruling): per-lane 1-bit mask, packed low in the `LINE_W` line (M5).
- **`FUNC_SEL` element-width field disposition** — retire the field from the register map, or keep it reserved for the fp8/integer growth modes.

2.9 9. Dependencies

- `rtl/math` `math_bf16_*` family — the implementation base (§3 names the block per row): adder, multiplier (`mantissa_mult`, `exponent_adder`), `fma`, `comparator`, `min`, `max`, `relu`, plus the activation, divide, scale, and conversion blocks of the growth list.
- `cell/docs/PRD.md` — the cell around the ALU (memories, FSM seam, packetizer, credits; reconciled to this PRD 2026-10-17).
- `kestrel-rv32i/` — the tetrad controller's reference core (falcon suite; `CORE_FLAVOR`-swappable per the HIVE-C precedent).
- `tile/docs/TILE_SPEC.md` §11 — packet, credit/grant, and coordinate protocols the cell (not the ALU) answers to; listed so a reader of this PRD can find the whole contract. (The tetrad-level control core is not yet recorded in the tile spec — flagged for its next edit.)
- `rtl/math/references/README.md` — the central math-paper index (bf16 format, activation, quantization, and fp8 citations for §§3, 7).

2.10 10. References

- `cell/docs/PRD.md` — the cell around the ALU (normative for everything except the ALU block, which this PRD owns).
- `riscv-ip/README.md` — the falcon suite (kestrel rung: the tetrad controller's reference core; `CORE_FLAVOR` precedent per the hive-serv PRD §10.3).
- S. Wang, P. Kanwar, "BFloat16: The Secret to High Performance on Cloud TPUs," Google Cloud blog, Aug. 2019 — the bf16 format's canonical definition (non-peer; citation-only in the math index). `rtl/math/references/README.md`.
- V. Nair, G. E. Hinton, "Rectified Linear Units Improve Restricted Boltzmann Machines," *Proc. 27th ICML*, 2010 — backs the opcode-7 ReLU row. PDF: `rtl/math/references/ReLU__Nair2010.pdf`.
- D. Hendrycks, K. Gimpel, "Gaussian Error Linear Units (GELUs)," arXiv, 2016 — backs the §7 gelu growth row. PDF: `rtl/math/references/GELU__Hendrycks2016.pdf`.
- S. Elfwing, E. Uchibe, K. Doya, "Sigmoid-Weighted Linear Units…" (*Neural Networks*, 2018) and P. Ramachandran, B. Zoph, Q. V. Le, "Searching for Activation Functions" (Swish, arXiv 2017) — back the §7 silu growth row. PDFs: `rtl/math/references/SiLU__Elfwing2018.pdf`, `rtl/math/references/Swish__Ramachandran2017.pdf`.
- B. Jacob et al., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," *Proc. IEEE CVPR*, 2018 — backs the §7 int8-drain row (`math_bf16_scale_to_int8`) and the §3 int8-lane roadmap. PDF: `rtl/math/references/Int8Quantization__Jacob2018.pdf`.
- D. Kalamkar et al., "A Study of BFLOAT16 for Deep Learning," arXiv, 2019 — backs the bf16 format choice (secondhand citation; primary provenance is the Google Cloud blog and the IEEE 754-2008 fp32 exponent). PDF: `rtl/math/references/README.md`.
- The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA — the custom-0 opcode space is the defined home for the gatling Xcell instruction subset (KICK/WAIT/…) described in §5. https://riscv.org/technical/specifications/

Last Updated: 2026-10-19 **Maintained By:** RTL Design Sherpa Project

← Back to cell

3 Cell — Compute Engine IP Product Requirements Document

Version: 0.5 (2026-10-19): records the 2026-10-19 owner ruling that **gatling sequences the cell ALU** (POP/PUSH/FLUSH Xcell encodings, gatling PRD v0.2) while the slice-3 `cell_exec_fsm` remains the standalone bring-up driver; DEST is explicitly reserved for the deferred cell→cell path and FLUSH_DEST/FLUSH_TAG are the operational drain destination; strap terminology replaces remaining grant terms in §§6.3/7/10 (slice-1 RTL still uses the old grant identifiers, noted in the MAS). Carries v0.4 (2026-10-18): §6.3 rewritten to the strap model (TILE_LAND_FLITS, RAPIDS returns on drain; owner direction 2026-10-18) and the result destination becomes gatling-programmed (TILE_SPEC §11.14) — DEST chain-back superseded, register retained for the deferred cell-destined path. Carries v0.3 (2026-10-18): **cell register file** + **sender-generic credits** — owner direction 2026-10-18 defines the cell as the lowest module: the alu-infer ALU, one delta node, the config registers, the ALU registers (A/B/C/D + T) as persistent operands for repeated calculations over a run (OP_SRC operand-source mux, IP/OP memory pointers), and the data-in / data-out memories; the tetrad engine is deferred (the owner-authored gatling-rv32itetrad barrel-core PRD is the leading candidate; §10). Credit returns become sender-generic — per-source batching to the recorded source coordinate; the packetizer gains DEST (default: recorded source, chain-back). §4 reconciles to the alu-infer elastic valid/ready pipeline (“one-function-at-a-time” superseded). The house-style graphviz figure replaces the ASCII block diagram. Carries v0.2: ALU element model, function set, and control interface reconciled to the alu-infer PRD v0.3 (owner direction 2026-10-17; owner datapath decisions 2026-10-16). **Status:** Pre-HAS PRD. This document is the cell’s product definition; the HAS/MAS follow it. The tile architecture it plugs into is the adopted hierarchy of `tile/docs/TILE_SPEC.md` §11 (owner decision 2026-10-16); the packet header, payload types, coordinate, and credit/grant protocols referenced below are defined there and are not restated except where the cell consumes them.

3.1 1. Identity and scope

The **cell** is the tile's compute unit: sixteen of them make a tile. Each cell contains exactly **one delta node** — a `delta_mesh_cell` instantiated inside the cell, its local port facing the cell's internal datapath and its seven other ports passing through to the cell boundary. That containment is the **entire** cell↔delta relationship (owner clarification 2026-10-16): the node is a black-box subcomponent governed by delta's own HAS/MAS, and everything else in the cell — the memories, the execution FSM, the ALU, the ALU registers, the packetizer, the credit logic, the registers — is the cell's own design, speaking the tile-level contracts of TILE_SPEC §11, not delta internals. The node fills the cell's input memory from delta traffic; the cell's ALU and execution FSM consume that memory and produce results; results leave as batched flits back through the node toward their destination. As each input line is consumed, a credit returns to the line's sender so it may write more.

Owner decision 2026-10-16: the cell's compute engine is the **ALU + execution FSM** described here. The earlier CGRA-array ambition (README, 2026-10-13) is **deferred** — §11 — and the `axon_spec.md` sketch (SERV-based result collector) is **superseded**: the result collection, packetization, and streaming it proposed are this PRD's output path, and its interfaces section predates the adopted §11 header/credit protocols.

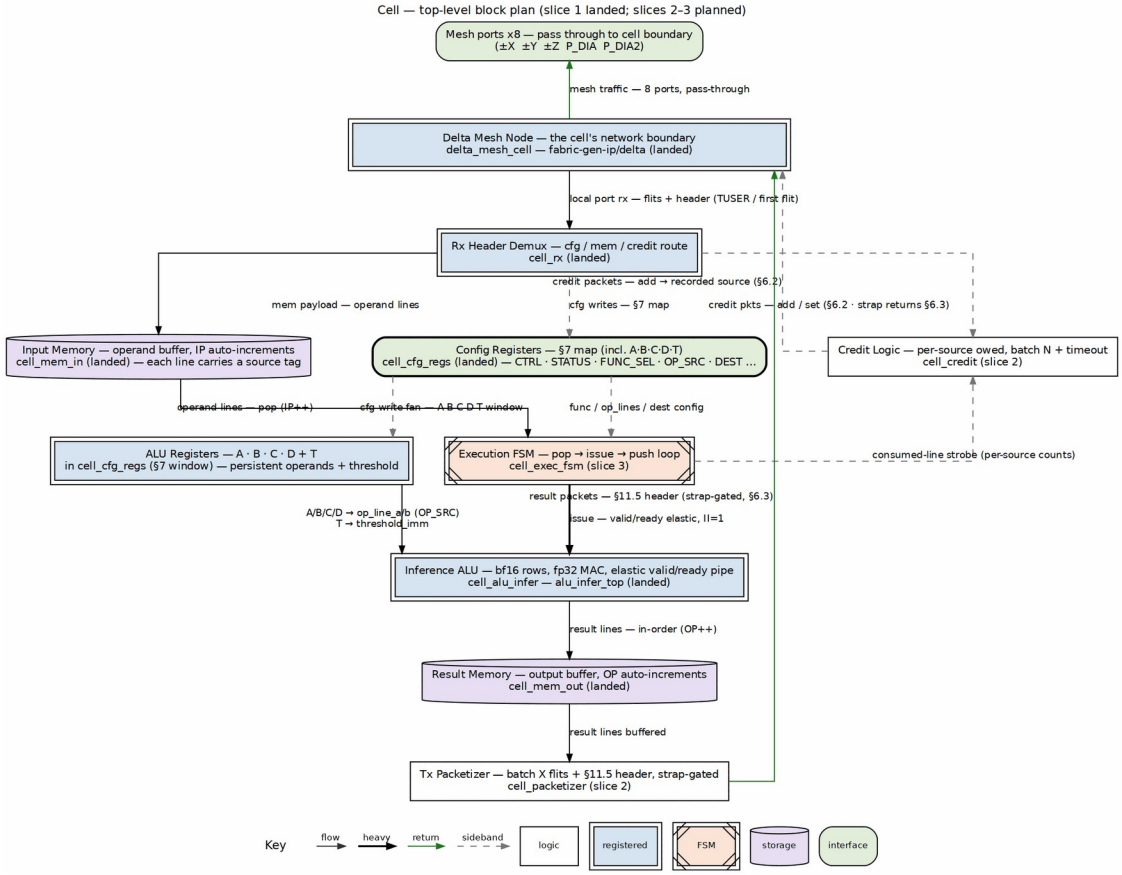
Owner direction 2026-10-17: the ALU block is governed by the `alu-infer` PRD — bf16 element model on the `rtl/math` `math_bf16_*` library, fp32 MAC accumulation, and a per-tetrad RV32I control core (not SERV; SERV is the Ocelus observability agent). Config and programming arrive in-band with the delta traffic; the tetrad core is a local reaction agent, not a config originator. See alu-infer PRD §5.

Owner direction 2026-10-18: **the cell is the lowest module and is defined first.** Its contents: the alu-infer ALU, one delta node, the config registers (§7), the ALU registers A/B/C/D (+T) — persistent operands for a sequence of the same calculations over a run — the data-in memory, and the data-out memory. The tetrad-level engine is **deferred**: gatling/tetrad is figured out once the cell is fully defined and working, with the owner-authored `gatling-rv32itetrad` barrel-core PRD (2026-10-17) the leading candidate. See §10.

Out of scope: tetrad/root node internals, rapids internals, the CXS modules themselves, and fabric-level Z stacking.

3.2 2. Architecture

3.2.1 Figure 1: Cell top-level block plan (house style — three flows: programming down the center, operand fill on the left, results and credits back up the right)



Cell top-level block plan

Block	Function	Notes
`delta_mesh_cell`	network attachment	instantiated black box, not owned, here — containment is the whole cell↔️delta relationship (owner clarification 2026-10-16); the cell consumes its local port and passes its mesh ports through; delta HAS/MAS

Block	Function	Notes
		govern only the node
rx header/demux	parses the §11.5 tile header (first data flit) and delta's route header (TUSER on AXIS), routes payload to input memory, handles cfg writes to registers, routes credit packets to the credit logic	per §11.6 (TILE_SPEC v1.6 clarification: tile header = first data beat on both transports; cell-bound traffic rides delta QoS ≠ 0 — QoS 0 is the node's own config plane); records each packet's tile-header source coordinate
input memory	delta-written operand/input buffer	sized in lines; its flit capacity is the boot advertisement value; each line carries a source-coordinate tag for credit returns (§6.2); read pointer IP auto-increments on pop
config registers	AXIL, registers-only per house rule	§7 — CTRL/STATUS/FUNC_SEL/OP_SRC/DEST/batching knobs/IP/OP
ALU registers	A·B·C·D persistent operands + T threshold	§7; written by cfg packets or AXIL, read into the ALU operand muxes per OP_SRC; persistent across a run (owner direction 2026-10-18)
credit logic	returns credits to the recorded sender of each consumed input line	§6.2; per-source batching, N configurable, default 1
ALU + execution FSM	pops line(s), issues them to the ALU's elastic	§4; the cell's only datapath

Block	Function	Notes
	pipeline, pushes result line(s)	
result memory	buffers result lines before packetization	sized with §12; write pointer `OP` auto-increments on push
packetizer	batches result writes into X flits, builds §11.5 header (destination per §6.3 — gatling-programmed drain), sends via node	sends only against the `TILE_LAND_FLITS` strap counter (§6.3)

3.3 3. Dataflow

A run is programmed first, then fed; everything — data, config, and programming — arrives in-band (owner ruling 2026-10-17):

1. **Program.** `cfg` packets (`ptype` = `cfg`) write the §7 registers — `FUNC_SEL`, `OP_SRC`, `DEST`, the persistent operands `A`–`D`, `T`, and the batching knobs — instead of the input memory. A `cfg` read is answered with a write-format packet echoing the tag, per §11.8.
2. **Fill.** A write packet (`ptype` = `mem`, destination coordinate `{level=0, cell X, Y, Z}`, target address inside the cell's input-memory window) arrives on the node's local port — from rapids in the v1 flow, but sender-generic by construction: any node may send it. The demux writes payload lines into the input memory and tags each line with its packet's source coordinate. The sender spends one end-to-end credit per flit sent, header included, from the per-destination counter it holds for this cell (§11.9).
3. **Consume.** The execution FSM pops the configured number of input lines for its function (`IP` auto-increments), issues the invocation to the ALU, and the elastic pipeline returns result line(s) in order; the FSM pushes them into the result memory (`OP` auto-increments).
4. **Credit return.** As each input line is consumed, the credit logic owes the line's recorded sender one credit (§6.2 — per-source batching), so the sender refills the space. Credits return on consumption, not arrival (§11.9(3)).
5. **Emit.** When the packetizer has ≥ X result flits (or a flush condition fires, §6.3), it builds the §11.5 header — destination = the gatling-programmed drain destination (§6.3; the cell does not choose it) — source = this cell's external ID, `op` = 0, length = bytes that follow, tag = the sink channel gatling programmed — and sends the packet through the node, spending its `TILE_LAND_FLITS` strap counter (§6.3). Cell-destined sending awaits the tile-spec extension (§6.3).

3.4 4. ALU and execution FSM

The FSM is a pop→issue→push loop: it issues invocations into the ALU's valid/ready **elastic pipeline** (multiple invocations in flight, results retire in order — the alu-infer MAS contract), and pushes each retired result line into the result memory. Functions are register-selected, and both the function and its operand pattern come from config, so new functions are RTL table rows, not new blocks. (The v0.2 "one-function-at-a-time" phrasing is superseded by the elastic contract; owner ruling 2026-10-17, alu-infer PRD v0.4+.)

Operand sources (owner direction 2026-10-18): the ALU's `op_line_a` / `op_line_b` are driven through an operand-source mux configured by `OP_SRC` (§7): each side selects the input-memory pop or one of the persistent ALU registers `A`–`D` (32 bits each, two bf16 lanes — matching `LINE_W`), and `T` (bf16) backs `threshold_imm`. This is the "sequence of the same calculations over a run" pattern: streamed operands arrive from delta into the input memory and pop under `IP`, while persistent scalars (weights, biases, constants) sit in `A`–`D` for the whole run. `IP`/`OP` auto-increment on pop/push and wrap at the configured memory depth; `CTRL.soft_reset` clears them.

Element model (owner direction 2026-10-17, superseding the B4/B8/B16/B32 note of 2026-10-16): a 32-bit line is read as **2×bf16** elements, and the function array is built mostly from the `math_bf16_*` family of `rtl/math` (adder, multiplier, FMA, comparator, min/max, ReLU, activations as growth). The ALU's product definition — element model, function set, interface contract, verification doctrine, and the per-tetrad control interface — is the **`alu-infer` PRD**, which governs the ALU block; it is not restated here (one source per fact). MAC accumulates in fp32 with one rounding per result line at drain. Wider lines (§12) scale the lane count, never the element width.

Gatling sequences the cell ALU (owner ruling 2026-10-19): in the tetrad-integrated tile, gatling (the barrel-core tetrad engine) drives the cell's alu-infer ALU through `Xcell` instructions: `POP` presents operand line(s) to the ALU issue handshake, `PUSH` retires result line(s) into the result memory, and `FLUSH` programs `FLUSH_DEST`/`FLUSH_TAG` and triggers the drain. The slice-3 `cell_exec_fsm` remains the standalone bring-up driver with the same POP/issue → PUSH/retire client contract. Gatling never originates cell config traffic; cell programming stays in-band per the 2026-10-17 owner ruling. The `Xcell` encodings (`CFG`, `POP`, `PUSH`, `ECTX`, `FLUSH`, `KICK`, `WAIT`) live in gatling PRD v0.2.

v1 function set: eight opcode rows (copy, add, sub, mul, mac, min/max, compare/threshold, relu) defined with their `math_bf16_*` implementation base in alu-infer PRD §3; not restated here. Opcode space reserves 5 bits; 8–31 are growth. Exact datapath widths and latency land in the MAS, consistent with the math area's learning ethos (sophisticated low-latency techniques).

3.5 5. Packet interface

The cell consumes and produces the §11.5 header unchanged; §11.6 locates it per transport. Fields the cell uses directly:

Field	Cell use
Destination coordinate	demux match against `{level=0, own X/Y/Z}`; rapids-bound emits use `FLUSH_DEST` (gatling-programmed). `DEST` is reserved for the deferred cell→cell path (TILE_SPEC §10).
Source coordinate	the tile header's source field (the originator — TILE_SPEC v1.6 §11.5 note; not delta's hop-source route header), recorded per arriving packet — credit-return destination and the deferred chain-back default for `DEST` (rapids sends as `level` 3, §11.3)
`ptype`	`mem` → input memory; `cfg` → registers; `credit` → credit/grant handler
`op`	write vs `cfg` read; credit `add` vs `set`
Length	payload bytes / credit count / read size
Target address	input-memory line index or register offset
Tag	echoed on replies and result packets so rapids routes them to the right sink channel

3.6 6. Credit behavior

3.6.1 6.1 Boot advertisement

After reset the cell sends one `set` credit packet (`op` = 1) to `level` 3 carrying its input memory's size **in flits**. Rapids's per-destination counter for this cell starts at 0 and cannot send until the advertisement arrives — the advertisement is the cell's "alive" (§11.10). The packet rides the credit VC and needs no grant.

3.6.2 6.2 Consumed-line credit returns

- Each input line consumed by the FSM owes **its sender** one credit return. The sender is the source coordinate recorded when the demux wrote the line (every input-memory line carries a source tag). Rapids (`level` 3) is the sender in the v1 flow, but the return is sender-generic per §11.9 — a credit return goes to the source coordinate of the packet whose flits it frees, so a cell that streamed into this cell's input memory receives the credits.
- Returns are `add` credit packets addressed to the recorded source coordinate, per §11.9(4): routed like any packet by destination coordinate; the sink port accepts them unconditionally and never stores them. They ride the credit VC and need no grant.
- **Batching (owner decision 2026-10-16, extended sender-generic 2026-10-18):** the credit logic keeps a per-source owed count and sends one credit packet per N consumed lines **per source**, with a timeout so a partial batch drains. **N = 1 is the default and the exact per-line behavior.** N > 1 trades return latency for less credit-packet traffic on the fabric.

3.6.3 6.3 Straps on the result path

The result path adopts §11.9's strap mechanism (owner direction 2026-10-18: credits toward RAPIDS are **straps, not grants**):

1. Toward RAPIDS the cell keeps no grant protocol. Space in RAPIDS's sink is fixed by construction: a per-cell strap of `TILE_LAND_FLITS` flits (a Hive-set parameter passed to RAPIDS, default 16 — TILE_SPEC §11.9 Hive contract), reserved at configuration.
2. The packetizer sends a result packet only while its strap counter covers the whole packet, header included, and spends the counter as it sends. RAPIDS returns `credit`/`add` as it drains the cell's landing space, restoring the counter.

3. Resync/re-baselining of the strap state follows RAPIDS's advertisement/resync rules (TILE_SPEC §11.10); exact timing is a tile §10 open item.
4. The cell never asks for result space — no ask path; the strap is the contract.

Result destination (owner direction 2026-10-18, TILE_SPEC §11.14): gatling hands the cell the drain destination. Result flushes go to `{ext 0, level 3}` (the tile I/O), source = this cell's external ID, tag = the sink channel gatling programmed. **The cell does not choose the destination.**

- **Operational drain:** `FLUSH_DEST` (coordinate) and `FLUSH_TAG` (sink channel) are programmed by gatling and used by the packetizer when `CTRL.flush-and-send` fires.
- **Reserved/deferred:** the legacy `DEST` register (offset `0x38`) is retained only for the future cell→cell result path. It is **not** used for rapids-bound drains. `DEST` becomes operational only after the tile-spec cell→cell credit/grant extension lands (TILE_SPEC §10); until then, v1 result traffic is RAPIDS-bound and `FLUSH_DEST`/`FLUSH_TAG` take precedence.

3.7 7. Config registers

AXI4-Lite, registers-only (house rule); routing/mode config travels in-band as `cfg` packets per the standing owner decisions — every register below is writable both from AXIL and from the `cfg` stream, and readable over AXIL for debug. v1 map:

Offset	Register	Notes
0x00	`CTRL`	enable, soft reset (also clears `IP`/`OP`), flush-and-send (packetizer)
0x04	`STATUS`	busy, input-empty, result-nonempty, strap counter nonzero (slice-1 RTL alias `status_grant_nonzero`)
0x08	`FUNC_SEL`	opcode (5b) + sub-op (2b) + reserved (2b) — element width is reserved under the bf16 model (alu-infer PRD §3)
0x0C	`OP_LINES`	input lines per function invocation
0x10	`EMIT_BATCH_X`	flits per result packet (packetizer)
0x14	`CREDIT_BATCH_N`	consumed lines per credit packet **per source**, default 1
0x18	`IRQ_EN`	done, error, strap-starved (slice-1 RTL alias `IRQ_GRANT_STARVED_BIT`), credit-timeout enables
0x1C	`IRQ_STATUS`	write-1-clear status
0x20	`A`	persistent operand register A — 32b (2×bf16 lanes); feeds `op_line_a/b` per `OP_SRC`
0x24	`B`	persistent operand

Offset	Register	Notes
		register B
0x28	C	persistent operand register C
0x2C	D	persistent operand register D
0x30	T	bf16 threshold immediate backing register (opcode 6)
0x34	OP_SRC	operand-source mux: sel_a[2:0], sel_b[2:0] — 0 = input memory (pop), 1 = A, 2 = B, 3 = C, 4 = D; reset 0 (both operands from memory)
0x38	DEST	*reserved for the deferred cell→cell path (2026-10-18/19 ruling)* — the rapids-bound drain destination is gatling-programmed into FLUSH_DEST/FLUSH_TAG; the cell does not choose it. DEST is retained (bits [13:0] = §11.3 coordinate at COORD_W = 4; bit [16] = override) but is **not** used until the tile-spec cell→cell credit/grant extension lands (TILE_SPEC §10).
0x3C	IP	input-memory read pointer — auto-increments on pop, wraps at depth
0x40	OP	result-memory write

Offset	Register	Notes
		pointer — auto-increments on push, wraps at depth
0x44+	counters	lines in, lines out, credits returned, packets sent (§8)

The A–D/T window is written by `cfg` packets like any other register (§3); loading A–D *from* the input memory is deferred (§11).

3.8 8. Status and observability

- IRQs: computation done, error, strap-starved (a packetizer with data and no strap, time-limited), credit-timeout.
- Software counters per §7.
- One MonBus observer stream per cell, on AXI4-Lite (owner direction 2026-10-19), per the tile's §5 aggregation hierarchy: the cell's stream is a leaf the tetrad-level merge consumes, alongside the other three cells and the tetrad node. Exact leaf selection is the tile §10 open item.

3.9 9. Verification hooks

cocotb + rds-dv BFMs per the standing convention. Test levels follow the house gate/func/full grid. Natural hooks: drive the node from a GAXI master instead of a mesh (the node's local port is a normal stream face), force credits and grants from the TB, and check the credit loop (count lines in → credits out, per recorded source) independently of the datapath.

3.10 10. Dependencies

- delta — the backbone network — **in progress**, built by a separate agent (fabric-gen-ip/delta). The cell instantiates one `delta_mesh_cell` node as a black box; modes 9–10 diagonal ports landed (Task 6c), the rest of the backbone is being built out.
- rapids — the DMA running data in and out of the tile — **in progress**, separate lane. The cell's dependencies on it are the per-destination credit counters, strap pool (`TILE_LAND_FLITS`), and sink-to-source credit-return path (§11.9 requirements list); sequencing with the rapids lane.
- rapids_cxs + delta CXS migration — **in flight** (amba lane); the cell's §11.6 transport note applies unchanged until delta ports move.
- TILE_SPEC §10 open items that touch the cell: input-buffer sizing, grant allocation policy, resync timing, cell→cell grant mediation (needed for cell-destined `DEST`, §6.3).
- the alu-infer PRD — the ALU block's product definition (bf16 element model, function set, interface contract, control interface); **normative for the ALU** since 2026-10-17.
- the alu-infer MAS — the implementation-intent book (module tree, function rows, accumulator, interface, cycle behavior, verification, configuration); lives in this directory's `docs/alu_infer_mas/`.
- a tetrad-level RV32I control core per alu-infer PRD §5 — **deferred** (owner direction 2026-10-18): the tetrad engine is defined once the cell is fully defined and working, with the owner-authored `gatling-rv32itetrad` barrel-core PRD the leading candidate and kestrel-class a candidate flavor via `CORE_FLAVOR`. When it lands, gatling sequences the cell ALU through `POP`/`PUSH`/`FLUSH` (owner ruling 2026-10-19); the cells see its local AXIL register plane and its operational writes only. Cell *programming* stays in-band (owner ruling 2026-10-17).

3.11 11. Deferred

- CGRA array of richer PEs (2026-10-13 ambition): the cell's block boundaries — node, input memory, compute, result memory, packetizer — are chosen so an array replaces the ALU+FSM block without disturbing the network, credit, or observation contracts.
- Context switching / multi-function pipelines.
- Cell-initiated grant requests.
- Wider-than-32-bit lines (element widths stay bf16-only regardless; §4).
- Loading `A`–`D` from the input memory (v1 writes them via `cfg`/AXIL).
- Cell-destined result sending (`DEST` = another cell) pending the tile-spec cell→cell grant extension (§6.3).
- The tetrad-level control core and the gatling/tetrad engine (§10).

3.12 12. Open questions

- Input memory and result memory depths (lines, and flit-conversion for the advertisement) — sized in the MAS against the MAS injection-FIFO and credit-flow analysis.
- `cfg` read reply path: whether rapids's sink half strips headers (TILE_SPEC §11.7 open item) affects nothing here — the cell always builds a full reply header.
- Flush conditions for the packetizer beyond batch-X (idle timeout? end-of-job marker from the FSM?).
- Cell→cell grant/advertisement rules: who grants sink space for cell-destined results, and how a cell advertises its input-memory credits to peer cells (today: to rapids only, §11.10). Needed for `DEST` overrides (§6.3); lands in the tile spec first.
- `DEST` coordinate width tracks the §11.3 `COORD_W` parameterization (4 today); the register is 32-bit with the used low bits defined above.

3.13 References

- `alu-infer/PRD.md` — the ALU block's product definition (normative for the ALU since 2026-10-17).
- `tile/docs/TILE_SPEC.md` — adopted tile hierarchy, §11 packet header, credits/grants (normative for §§3, 5, 6).
- `delta_has/ch07_mesh_network/` — the node the cell instantiates.
- `riscv-ip/gatling-rv32itetrad/PRD.md` — owner-authored barrel-core tetrad-engine PRD (leading candidate; deferred per §10).
- `axon_spec.md` — superseded 2026-10-16 (decision trail).
- `2026-10-13-delta-mesh-design.md` — working design record.

Last Updated: 2026-10-19 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

4 alu-infer — Cell Inference ALU (definition)

The cell's compute ALU: a bf16 inference datapath — two bf16 lanes per 32-bit line — built mostly on the repo's `math_bf16_*` library (adder, multiplier, fused FMA-mac, comparator, min/max, ReLU, and the activation growth set), driven by the cell's execution FSM.

This directory holds the **product definition and the RTL**; the ALU is integrated and verified at system level inside the cell component (owner decision 2026-10-17; RTL moved here from `cell/rtl/` on 2026-10-20).

- **PRD:** PRD.md
- **MAS:** Micro-Architecture Specification (PDF) — module tree, function rows, fp32 MAC accumulator, ALU↔FSM interface, cycle behavior, verification hooks
- **RTL:** `rtl/` — `alu_infer_top.sv`, `alu_infer_rows.sv`, `alu_infer_pipe.sv`, `alu_infer_acc.sv`, organized like the other component areas (per-block filelists under `rtl/filelists/`, an `alu_infer_all.f` lint master, four-line area Makefile)
- **Unit tests:** `dv/tests/` — `test_alu_infer.py` (func+gate green), run via `make -C dv/tests run-alu_infer-<level>`
- **Integration home:** `cell/` — the "ALU + execution FSM" block; the normative datapath context is cell PRD §4. The cell filelists include this block's filelist for slice-3 instantiation.
- **Arithmetic library:** `rtl/math/` `math_bf16_*` family
- **Back to family:** compute-eng-ip README

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

← Back to tile

5 Tile — Compute Engine Tile Subsystem Specification

Version: 1.8 (2026-10-19; §11.13/§11.14 tetrad control core and result flush — owner direction: gatling sequences the four alu-infer ALUs of its adjacent cells via POP/PUSH against the cell ALU issue/result contract, the cell slice-3 exec FSM stays the standalone bring-up driver with the same client contract, and gatling never originates config traffic; Xcell FLUSH programs the cell's FLUSH_DEST/FLUSH_TAG and triggers flush-and-send; §12 milestone ladder M1–M4 added). Carries 1.7 (2026-10-19; §5 MonBus per level — owner direction: every cell emits one MonBus on AXI4-Lite, the tetrad merges every MonBus at its level (4 cells + the tetrad node) into one AXI4-Lite out, and the tile merges the 4 tetrads and the tile-level nodes (root 0, root 1) into one MonBus). Carries 1.6 (2026-10-18; §11.5–11.6 header-transport binding — two-header model: delta route header in TUSER (AXIS), the 128-bit tile header as the first data flit on both transports per the RAPIDS built implementation; §11.2 cell-traffic QoS rule — QoS 0 reserved for the node's own config plane, all cell-bound traffic rides QoS ≠0; TILE_LAND_FLITS home is the Hive top. Clarifications, no mechanism change. Carries 1.5 (2026-10-18; §11.13 tetrad control core **deferred** pending the cell — owner direction 2026-10-18, gatling-rv32itetrad the leading candidate; cell→cell grant mediation added to §10 open items; §11.3 internal/external ID on every node, §11.9-11.10 credits — owner direction 2026-10-18: delta nodes are conduits for transit and endpoints for what they consume, 32 RAPIDS credit counters, the advertisement window's saved legal-destination mask, and a cell's advertisement sent by its delta node from a parameter, then (same day) delta nodes only transport and never originate: a cell or core advertises for itself, nodes are legal by topology, and straps toward RAPIDS replace grants; §11.14 result flush to the tile I/O, delta must carry result data to level 3). Carries 1.4 (tetrad control core recorded 2026-10-17; §11 adopted 2026-10-16; cell integration decisions 2026-10-16; §11 proposal added 2026-10-16; re-homed to compute-eng-ip 2026-10-15; v1.0 owner-defined 2026-10-15) **Status:** Canonical microarchitecture definition for the compute-engine tile. **delta is the node** (delta_mesh_cell) — nodes collectively form the network; this document defines the tile IP that instantiates 16 of them. Where this document and delta_has/ch07_mesh_network/01_hierarchy.md disagree, this document wins — the hierarchy chapter predates the 2026-10-15 owner decisions (tetrad/tile naming, real wrapper modules, MonBus aggregation, this re-home). The working design record with full citations is docs/superpowers/specs/2026-10-13-delta-mesh-design.md (§2 in particular); this spec is its distilled, decision-frozen form.

5.1 1. Scope

The **tile** is the delta mesh's subsystem unit: 16 cells arranged as 2×2×1 **tetrads** of 2×2×1 cells, fully connected at both scales, with per-level MonBus observation aggregation and a single external attach point for rapids. This document defines the tile's hierarchy, modules, connectivity, observation architecture, interfaces, and mode set.

Out of scope: cell-internal microarchitecture (buffering, allocators, VCs — design spec §3 and ch07 §§4–6), flit format (§4), the flat crossbar modes of the delta generator's other configurations, and the fabric-level Z stacking above one tile.

5.2 2. Hierarchy

Level	Composition	RTL	Purpose
Cell (node)	1	`delta_mesh_cell` — owned by `fabric-gen-ip/delta`	the one module with datapath: routing tables, VCs, allocators
Tetrad	2×2×1 cells (4)	`tetrad` (this IP)	datapath grouping (K4) + first MonBus aggregation
Tile	2×2×1 tetrads = 4×4×1 cells (16)	`tile_top` (this IP)	the subsystem: full K4 at tetrad scale, tile-top MonBus, rapids attach
Fabric	4×4×K tiles	generated top	planar recursion; Z lives between tile layers

Owner-named 2026-10-15: “tetrad” (was “tile”) and “tile” (was “confluence”).

Adopted 2026-10-16: §11 refines this hierarchy — the tetrad is a routing node with datapath (not a passive wrapper), two root nodes sit above the tetrads behind a root mux, and rapids attaches at the tile I/O above the mux (§11.1, §8 below). The table’s RTL column names predate that decision; the wrapper-shaped `tetrad`/`tile_top` plan built on it is superseded. The hierarchy’s leaves are the cells, defined by the cell PRD (`compute-eng-ip/cell/docs/PRD.md`) — each a self-contained IP whose only tie to delta is one node instantiated inside it.

Owner direction 2026-10-17: each tetrad additionally carries **one RV32I control core** (kestrel-class, `CORE_FLAVOR`-swappable) at its tetrad node’s local port — the control agent for the tetrad’s four cells, defined with its duties and boundaries in alu-infer PRD §5. All data, config, and programming for the tile blocks — the core’s firmware included — arrive in-band on delta; the core is a local reaction agent, not a config originator. SERV is unchanged in its Ocelus observability role. See §11.13.

5.3 3. Connectivity

Connectivity is the **same K4 clique at both scales** — the owner's figure `assets/3d-mesh-tile.drawio` draws it: four fully meshed cells inside the tetrad, four fully meshed tetrads inside the tile.

- **Cell ports**: 6 neighbors (±X, ±Y, ±Z) + 1 local (host/rapids-facing)
  - P_DIA + P_DIA2 + 1 MonBus observer out.
- **Intra-tetrad K4**: the 4 perimeter edges ride ordinary ±X/±Y links; both diagonals ride **P_DIA**, one link per cell to the diagonally opposite cell of its tetrad.
- **Intra-tile K4**: the 4 tetrads of a tile are themselves fully connected. Edge-adjacent tetrads share ±X/±Y boundaries; diagonal tetrad pairs ride **P_DIA2**, joining the diagonally adjacent corner cells (e.g. (1,1,0)↔(2,2,0), (1,2,0)↔(2,1,0) per tile). Out-of-grid mates tie P_DIA2 off — provably never selected.
- **One conceptual diagonal axis**: P_DIA and P_DIA2 are two instances of a single diagonal axis realized at tetrad and tile scale. A cell has 4 axes in-plane-plus-diagonal plus Z: 8 directional links + local = 9 ports.
- **Z**: entirely inter-tile; tetrads are planar (X/Y only).

5.4 4. Modules

- `delta_mesh_cell` — the node, owned and emitted by delta (the fabric-gen-ip component). The only datapath module: routing tables, VCs, allocators. The network is the collective of instantiated nodes; this IP is not it.
- `tetrad` (`rtl/tetrad.sv`, this IP) — instantiates 4 delta nodes; wires the K4 connectivity of §3; merges every MonBus stream at its level (the 4 cells and the tetrad node, §11.1) into one tetrad MonBus on AXI4-Lite (§5). No datapath function.
- `tile_top` (`rtl/tile_top.sv`, this IP — house `<ip>_top` naming) — instantiates 4 tetrads; merges the 4 tetrad MonBus streams and the tile-level nodes' streams (root 0, root 1) into the single tile-top MonBus (§5); exposes boundary nodes' local ports for rapids attachment. No datapath function. (Working names during the delta-side design phase were `delta_mesh_tetrad` / `delta_mesh_tile`; retired with this re-home.)

Adopted 2026-10-16 (§11.1): the tetrad becomes a routing node with datapath, so its module owns switching above its group of 4 cells; the tile adds 2 root nodes, a root mux, and the single tile I/O. The wrapper-only description above is kept for the decision trail; module boundaries for the adopted shape are part of the tile RTL planning and follow the cell PRD's block structure.

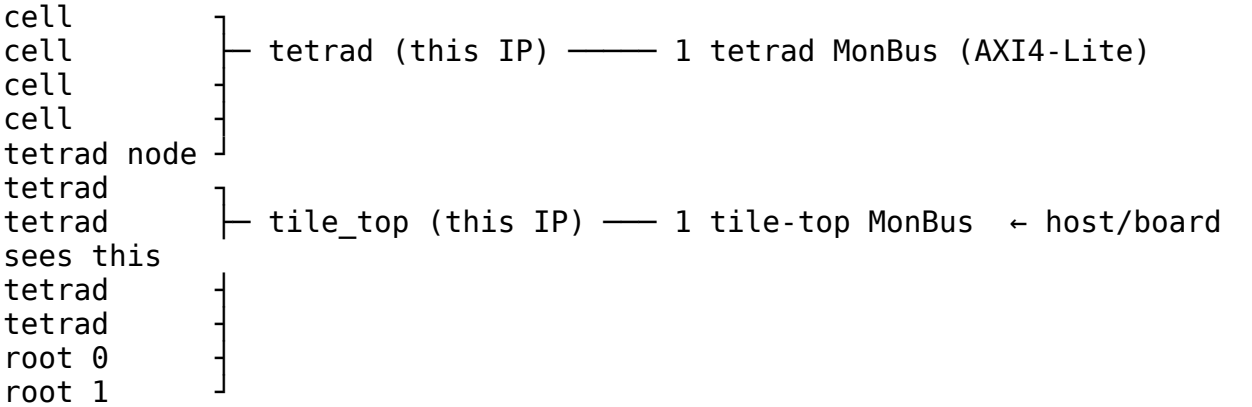
Owner direction 2026-10-17: the tetrad module also instantiates the tetrad's **control core** — one kestrel-class RV32I core (this IP) attached at the tetrad node's local port, per §11.13 and alu-infer PRD §5.

5.5 5. Observation (MonBus aggregation)

Owner decision 2026-10-15; refined by owner direction 2026-10-19. One MonBus leaves each level, and each level merges **every** MonBus source at that level, nodes included, not only the level below:

MonBus sources per level

Level	MonBus sources merged	Out
Cell	the cell	one MonBus, on AXI4-Lite
Tetrad	its 4 cells + the tetrad node (+ the control core, §11.13, when it exists)	one MonBus, on AXI4-Lite
Tile	the 4 tetrads + the tile-level nodes: root 0, root 1 (the root mux is not a node, §11.1)	one MonBus



Aggregation uses the house AXIL MonBus group family (`rtl/amba/ocelus/monbus_group_core` + `monbus_*_group` wrappers, AXIL host-read flavor) — the same observation path as the amber tally harness (merge → group → AXIL read-out), so standard host tooling (`monbus_tally_axil`, regmaps) attaches at the tile top unchanged. Exact leaf selection inside the wrappers is implementation detail; the architectural contract is **one MonBus per level boundary, merging every source at that level**: 5:1 at the tetrad (6:1 once the control core exists), 6:1 at the tile.

This hierarchy is also the verification anchor: per-tetrad and per-tile observability come for free, and the tile-top stream is the single sim-vs-silicon equivalence point (the same role the UART-equivalence cell plays for the amber harness).

5.6 6. Interfaces

- **Cell/tetrad/tile datapath ports**: AXI4-Stream (`TDATA/TVALID/TREADY/TLAST` + header sideband in `TUSER`), with the optional AMBA 5 TSTRB/TWAKEUP/TPARITY richness carrying over from flat mode unchanged.
- **CXS migration (owner plan, 2026-10-15)**: rapids gains a CXS interface variant — a separate model, `rapids_cxs`, replacing its AXIS interfaces — and delta's network ports move to CXS to match. Sequencing: rapids_cxs first (a working AXIS model exists to diff against), then delta. Header/sideband content is unchanged by the migration. Until both sides land, all delta ports stay AXI4-Stream.
- **MonBus ports**: per §5; cell-level streams are internal to the tetrad, tetrad-level streams internal to the tile.

5.7 7. Modes

The mode catalog (design spec §5.4) is unchanged by this spec; the tile-relevant rows:

Mode	Pattern	Tile relevance
1 Mesh (default)	DOR over RAM tables	baseline; inter-tile dsts DOR
9 Tetrad all-to-all (K4)	perimeter ±X/±Y + P_DIA diagonals	intra-tetrad 1 hop to any cell; inter-tetrad DOR
10 Tile all-to-all (K4 of tetrads)	shared boundaries + P_DIA2 diagonals	any cell of any tetrad of the tile in 1–2 hops (diagonal-of-tetrad pairs 1 hop); inter-tile DOR

Modes are table images, not new hardware — except that modes 9–10 make P_DIA/P_DIA2 (emitted in every build) their enabling hardware.

5.8 8. Rapids attachment

Hive configuration (owner, 2026-10-15): one or more tiles — only one for now — connect to a **single** rapids DMA at boundary cells' local ports; any boundary cell can be the attachment. With the CXS migration (§6), this link is the first CXS hop.

Adopted 2026-10-16 (§11.1): rapids attaches above the two root nodes through the root mux, at the single tile I/O — not at a boundary cell's local port. Its descriptors, buffers and channel state survive a root failover (§11.4); the cell↔rapids protocols (packet header, end-to-end credits, straps) are §§11.5–11.10.

Status (owner, 2026-10-18): rapids is the DMA running data in and out of the tile; RTL **in progress** on a separate lane, alongside the delta backbone (an agent is building that).

5.9 9. Clocking and reset

Single clock domain, active-low async `aresetn` (house `ALWAYS_FF_RST`) throughout the tile — a deliberate simplification against the NoC literature's GALS assumption; timing closure and generator predictability win over clock flexibility (design spec §2 close).

Resets and partial resets (a tile, a tetrad, RAPIDS or one of its channels) happen only during quiescent periods, under the global quiescent signal (§11.9, hazard rules). How each partial reset sequences against the credit state is still to be worked out (§10).

5.10 10. Open items

Decided 2026-10-16 (was §11): the hierarchical tile of §11.1 (tetrad routing nodes, two roots, root mux, tile I/O); the packet header and payload types (§11.2, §11.5); the destination coordinate (§11.3); end-to-end credits with boot advertisement, per-destination counters, and straps gating cell→rapids traffic (§§11.9–11.10, owner direction 2026-10-18, replacing grants); credit-return batching as a per-cell config with default N=1 (cell PRD §6).

Still open (see the milestone ladder in §12):

- The global quiescent signal (§11.9): how it is formed across every cell and tile, and how each reset and partial reset sequences against it.
- What switches the root mux (§11.4): software, a watchdog, or the root-to-root link.
- Cell input buffer sizes (the advertisement value) and result-memory sizing — cell PRD §12.
- Resync timing: how long RAPIDS waits for answers before declaring a destination dead (§11.10).
- Cell-issued descriptors (§11.11): whether a source channel is reserved for them or any channel may take them, and whether a cell is told when its pull completes.
- CXS leaf selection for both rapids_cxs and delta ports (§6).
- MonBus wrapper leaf selection inside tetrad/tile (§5).
- Config/address region naming per level — design spec §5, ch07 §3; §11.3's coordinate is the addressing answer, register naming follows the cell PRD.
- Cell→cell flow control (§11.9): credit *returns* are sender-generic already — a credit goes to the source coordinate of the packet whose flits it frees — but straps are defined only toward RAPIDS. Who reserves space for cell-destined results, and how a cell advertises its input-buffer credits to peer cells, is undefined. Needed for the cell PRD's `DEST` override (cell PRD v0.3 §6.3); v1 result traffic is rapids-bound.
- Result flush (§11.14): the gatling-to-cell command is now defined as the `Xcell FLUSH` custom instruction (gatling PRD v0.2); the cell's flush-and-send CSR details and encoding land in the cell/tetrad MAS.
- Tetrad control core (§11.13): the 2026-10-19 ruling records gatling sequencing the four cell ALUs and no config origination; the `KICK`/`WAIT`/`FLUSH` encoding, local memory map, and program-load conventions remain cell/tetrad MAS work. **Deferred** (owner direction

2026-10-18): the engine is defined after the cell is fully defined and working; gatling-rv32itetrad is the leading candidate (§11.13).

- Multi-path routing and failover (§11.15, delta DELTA-013): fully adaptive or pinned forward with an adaptive return; the transaction ID that makes ordering a requester-side property.

Tile RTL (`tetrad` node, `tile_top` with roots + mux + I/O) is pending the adopted-shape module split above; the node RTL (`delta_mesh_cell`, modes 9–10 diagonal ports, Task 6c) has landed in `fabric-gen-ip/delta`.

5.11 11. Hierarchical tile, root nodes, addressing (ADOPTED 2026-10-16)

Status: ADOPTED, owner decision 2026-10-16. Where §§2–4 and §8 still carry the pre-adoption wrapper wording, this section is the decision record and wins. Residual open sub-items are listed in §10.

5.11.1 11.1 Hierarchy

Level	Count per tile	Role
Cell	16, in 4 groups of 4	Leaf node; the 4 cells of a group are fully meshed (K4)
Tetrad node	4	A routing node above its group of 4 cells; the 4 tetrad nodes are fully meshed (K4)
Root node	2 (root 0, root 1)	A routing node above the tetrads; each root links to all 4 tetrad nodes; the two roots are linked to each other
Root mux	1	Selects the live root; not a node and not addressable
Tile I/O	1	The single connection out of the mux, to the one RAPIDS: its **source port** (RAPIDS to tile) and its **sink port** (tile to RAPIDS)

Changes against §§2-4: - The tetrad becomes a routing node with datapath, not a wrapper. - RAPIDS attaches above the roots, through the mux, not at a boundary cell’s local port.

5.11.2 11.2 Payload types

Every packet carries a payload type, chosen by the RAPIDS descriptor that produced it:

Code	Type	Target
0	`cfg`	Node or cell configuration (routing tables, mode, CSRs)
1	`prog`	Program load
2	`mem`	Node or cell memory
3	`credit`	End-to-end credit advertisement or return (§11.9); header only
4	`desc`	A cell-issued RAPIDS source descriptor (§11.11)
5-7	reserved	-

Delta QoS classes (owner clarification 2026-10-18): QoS 0 is **reserved for the delta node's own config plane** — a node consumes any head flit with `dst == POS && qos == 0` as route-table config and never ejects it to its local port. All cell-bound traffic (`cfg`, `prog`, `mem`, `credit`, `desc`) therefore rides delta QoS ≠ 0: class 1 (latency) for `cfg` / `credit` / `desc`, class 2 (bulk) for `mem` / `prog`. A cell's demux dispatches by the tile header's `ptype` (§11.5), never by QoS.

5.11.3 11.3 Destination coordinate

Proposed destination coordinate, `{ext, level, X, Y, Z}`, 15 bits at COORD_W = 4

Field	Width	Meaning
`ext`	1	Which of the node's two IDs (owner direction 2026-10-18): 0 = internal, the node itself; 1 = external, what is attached at its local port
`level`	2	0 = cell-level node, 1 = tetrad node, 2 = root node, 3 = tile I/O (RAPIDS)
X	`COORD_W` (4)	Cell: its X. Tetrad: its group's X (the cell X with the low bit dropped). Root: root 0 or 1. Tile I/O:

Field	Width	Meaning
		reserved.
Y	COORD_W (4)	Cell: its Y. Tetrad: its group's Y. Root and tile I/O: reserved.
Z	COORD_W (4)	Tile layer, unchanged from delta

- **Every node has an internal and an external ID** (owner direction 2026-10-18). The internal ID (ext = 0) is the node itself: its routing tables, its configuration, the buffer for packets addressed to it. The external ID (ext = 1) is what is attached at its local port: at a cell-level node, the cell; at a tetrad node, its control core (§11.13).
- A cell is therefore the external ID of its cell-level node, {1, 0, X, Y, Z}. Tetrad nodes, roots and the tile I/O are addressed by their own level, never as a cell.
- Only a node with something attached has external credit (§11.9): an external ID with nothing behind it never advertises, so it is never a legal destination.
- Routing uses {level, X, Y, Z} only; ext is read at the last hop, to consume the packet in the node or deliver it out of the local port.
- The mux has no coordinate.
- A packet's source coordinate uses the same format. RAPIDS always sends as level = 3, the tile I/O (ext = 0), so replies reach it through whichever root is live.
- Delta's head-flit header carries {X, Y, Z} today (ch07 §2). Adding level and ext is a delta header change, still to be made. It is required: delta must deliver result data to level 3 (§11.14).

5.11.4 11.4 Root failover

- Root 1 takes over if root 0 fails. There is one RAPIDS, so its descriptors, buffers and channel state survive a root failure.
- Packets in flight inside the failed root are lost. RAPIDS sees those transfers stall and time out (its scheduler timeout), and software re-issues them.
- The mux switches only at a packet boundary. On the CXS link to RAPIDS, a switch is a link deactivation followed by a reactivation, which the CXS

endpoints already handle (credits are returned before deactivation completes).

- Open: what drives the mux select (§10).

5.11.5 11.5 Packet header

Every packet starts with one header. It extends delta's head-flit header (delta HAS ch07 §2: coordinates, QoS class, VC, multicast, misroute budget, source-route pointer, packet ID) with the fields below.

Proposed tile packet header fields (beyond delta's existing ones)

Field	Width	Meaning
Destination coordinate	15	`{ext, level, X, Y, Z}`, §11.3; adds `ext` and `level` to delta's `{X, Y, Z}`
Source coordinate	15	Same format; RAPIDS sends as `level` = 3, the tile I/O
`ptype`	3	Payload type, §11.2
`op`	1	Data packets: 0 = write (payload follows), 1 = read (header only; the target replies). `credit` packets: 0 = add, 1 = set (§11.10)
Length	32	Write: payload bytes that follow. Read: bytes requested. Credit: flits (or descriptor entries) advertised or returned. `desc`: 32 (one descriptor).
Target address	49	Byte address inside the addressed node or cell. Credit packets: bits [1:0] are the credit class (0 = sink space, 1 = descriptor slots), the rest reserved.

Field	Width	Meaning
Tag	4	The RAPIDS channel. Set by RAPIDS on a request; echoed by the target in its reply, so the reply reaches the right sink channel

Proposed packing, for a transport that carries the header as data:

Proposed header packing

Bits	Field
`[14:0]`	Destination coordinate
`[29:15]`	Source coordinate
`[32:30]`	`ptype`
`[33]`	`op`
`[65:34]`	Length
`[114:66]`	Target address
`[118:115]`	Tag
above `[118]`	Delta's existing head-flit fields, then reserved

Two headers, two owners (owner clarification 2026-10-18): this tile header is a **payload** header — the first data flit of the packet. It is distinct from delta's **route header**, which delta owns and which rides `TUSER` of the head beat on AXI4-Stream (§11.6). Consumers that need the packet's *originator* — credit returns (§11.9), a cell's recorded source — read the tile header's source coordinate; delta's route-header `src` is the previous hop, not the originator.

The header costs one flit per packet. For short `cfg` writes that is a large share; a later revision could pack a small payload into the reserved bits.

5.11.6 11.6 Transport

The header is the same whatever carries it — a **data flit**, per the RAPIDS built implementation (`rapids_pkg::tile_hdr_pack`, 128 bits; see §11.12). Delta's route header (§11.5 note) is separate and stays delta's:

Transport	Delta route header	Tile packet header (§11.5)
AXI4-Stream (delta ports today)	`TUSER` of the head beat, as delta does now	First data beat of the packet (`tdata[127:0]` of

Transport	Delta route header	Tile packet header (§11.5)
		beat 0)
CXS (RAPIDS CXS tops; delta ports after the migration in §6)	Per delta's CXS port mapping (delta migration, §6)	The data of the first flit of the packet

Binding (owner clarification 2026-10-18): the earlier "AXIS: `TUSER` of the head beat" wording for the *tile* header is superseded — on AXIS, `TUSER` carries only delta's route header. Until delta ports move to CXS, a cell on an AXIS local port sees: `TUSER` = delta route header, beat-0 `tdata` = tile header, then payload beats.

Beat-chained header (owner ruling 2026-10-18): every link keeps its own data width; the 128-bit tile header occupies the first `ceil(128/DW)` payload beats of the packet, LSB-first (on a 64-bit link: beat 0 = header `[63:0]`, beat 1 = header `[127:64]`). No link widens and no width converter is needed anywhere in the fabric for the header's sake; the tile I/O link (256-bit) carries header and payload together.

On CXS the link's own sidebands keep their link-level meaning: RAPIDS keeps the channel in `CXSSRCID`, so per-stream framing is unchanged.

5.11.7 11.7 Where the fields come from

- **RAPIDS:** its descriptor supplies `ptype`, `op`, the destination coordinate and the target address (RAPIDS HAS, Descriptor Format); the tag is the channel that issued the descriptor. The source half builds the header and sends it out of the source port, ahead of any payload.
- **Nodes:** none. A delta node never originates a packet (§11.9), so it never answers a read; a cell or core answering one builds a reply header addressed to the request's source coordinate.
- **Open:** whether the RAPIDS sink half strips the header or delivers it to memory.

5.11.8 11.8 Operations

Only cells and control cores send into RAPIDS's sink port, and only against their straps (§11.9); a delta node never does.

Proposed operations between RAPIDS and the tile

Operation	Source port sends	Target does	Sink port receives
Write	Header (`op` = 0, length N, tag) + N bytes	Consumes the data, then returns credits (§11.9)	The credit packet

Operation	Source port sends	Target does	Sink port receives
Read	Header only (`op` = 1, length N, tag = the sink channel that will take the data); never to a node	Sends N bytes back: a write-format packet to `level` 3 with the tag echoed	The data, into the tagged sink channel
Cell send	-	Sends a write-format packet to `level` 3 with the tag set, against its `TILE_LAND_FLITS` strap	The data, via the cell's landing space, into the tagged sink channel
Credit return to a sender	Header only (`ptype` = `credit`, destination = the cell or core) for what RAPIDS consumed	Adds it to its strap counter	-
Credit return	-	-	Header only (`ptype` = `credit`) for flits the cell or core consumed; never stored in a channel
Core-issued descriptor	-	Sends a `desc` packet to `level` 3: header + one 256-bit source descriptor, tag = the source channel, against its strapped descriptor-queue entries	The descriptor, into that source channel's descriptor queue (§11.11)

A read carries its own reservation: RAPIDS issues it only when the tagged sink channel has room for N bytes, and the reply needs no strap credit.

5.11.9 11.9 End-to-end credits

Link credits (CXS `CRDGNT` / `CRDRTN`) cover one hop and carry no address, so they cannot say anything about a cell's buffer. End-to-end credits are packets instead.

delta nodes only transport (owner direction 2026-10-18). A node carries every packet on the data network, and `credit` packets on their own interface, and holds no credit state for it: only the hop-level flow control it already has. A node never originates a packet on the data network; whatever it generates itself (debug, status) goes on MonBus. It consumes what is addressed to its internal ID (§11.3) on arrival, so it needs no end-to-end credit: it never advertises and never returns credits, and it must never stall what is addressed to it. End-to-end credits belong to what is attached at a node's local port, under the node's external ID: a cell, or a tetrad's control core (gatling).

1. RAPIDS's source half keeps **64 credit counters** (owner direction 2026-10-18), one per destination index `{ext, n}`: the internal and the external ID of every node. A tile uses at most 42 (22 internal: 16 cell-level nodes, 4 tetrad nodes, 2 root nodes; 20 external: 16 cells, 4 tetrad control cores). An external counter starts at 0 and is filled by its cell's or core's own advertisement (§11.10), so an external ID with nothing attached is never legal. An internal (node) index is legal by topology: a packet to a node is never held for credits and never charged.

Index	Destination
0-15	Cell-level node (internal), `{Y[1:0], X[1:0]}`
16-19	Tetrad node (internal), `{Y[0], X[0]}` of its group
20-21	Root node 0, 1 (internal)
22-31	Spare
32-47	Cell: the cell-level node's external ID, `32 + {Y[1:0], X[1:0]}`
48-51	Tetrad control core: the tetrad node's external ID
52-53	Root external: none (the root's local side is the tile I/O)
54-63	Spare

: Proposed credit-counter index

2. Each flit sent to a destination, the header included, spends one credit. RAPIDS sends a packet only when the counter covers all of it.

3. The cell or core returns credits when it has **consumed** the flits (written them to memory, or applied the `cfg`), not when they arrive. It may batch them, for example one return per packet.

4. The return is a `credit` packet to `level` 3 with the count in the length field. The sink port recognises it, never stores it, and passes {source coordinate, count} to the source half, which adds the count to that destination's counter.

Counters are per destination, not per channel: two channels writing the same cell share that cell's buffer. Traffic routed through a node never touches a counter. A node never replies, so a read request to a node's internal ID is refused as an invalid descriptor; node registers are read over MonBus or AXIL.

Where a credit goes: a credit packet is routed like any packet, by its destination coordinate. A credit return goes to the source coordinate of the packet whose flits it frees, so a cell returning credits to RAPIDS always addresses `level` 3, and a credit return from RAPIDS goes to the cell or core whose packet it consumed.

Credits toward RAPIDS: straps, not grants (owner direction 2026-10-18). A cell or core must never send into the sink port without guaranteed space: flits that cannot drain stall in the routers and block everything behind them, credit packets included. The space is fixed by construction instead of granted at run time:

Credits toward RAPIDS

Sender	Sends to RAPIDS	Credit
Cell (a cell-level node's external ID)	Memory writes (results, §11.14)	A strap of `TILE_LAND_FLITS` flits (a Hive-set strap passed to RAPIDS, default 16; §11.9 Hive contract below), header included
Control core (a tetrad node's external ID)	`desc` packets (§11.11)	Entries in RAPIDS's `desc` intake queue
delta node	Nothing	None

1. Each sender starts from its strap after reset and sends a packet only while its counter covers the whole packet.
2. RAPIDS reserves every strap: a landing space of `TILE_LAND_FLITS` flits per cell, and a `desc` intake queue as deep as the descriptor credits handed out. The sink cannot be overcommitted however many senders send.
3. RAPIDS returns credits as it consumes, with a `credit`/`add` packet to the sender: flits when the data moves out of the landing space into its sink

channel, one entry when a descriptor moves from the intake queue into the descriptor queue. These returns are required, not optional.

The landing space is per sender: a cell whose data targets a full sink channel holds only its own landing space, never another sender's. A packet to RAPIDS is therefore at most `TILE_LAND_FLITS` flits.

With end-to-end credits in both directions, every packet in the fabric has guaranteed space at its destination and always drains.

Requirements this places on the implementation: - **Deadlock:** a credit must never wait behind data that is itself waiting for credits. Credits have their own interfaces (owner 2026-10-18), separate from data, so this holds by construction. - **RAPIDS:** the sink-to-source credit path inside the core, the 64-entry counter table (filled by the advertisements), the advertisement window and the saved legal-destination mask (§11.10); and toward RAPIDS, the `TILE_LAND_FLITS` landing space per cell, the `desc` intake queue, and the credit returns. - **Cells and control cores:** buffer sizing, their own boot advertisement, return batching, building credit packets, and a strap counter toward RAPIDS. - **delta nodes:** none of it. No credit state, no advertisement, no returns; they consume what is addressed to them without stalling. `credit` packets travel on their own interface.

Hazard rules (owner direction 2026-10-18): - **Always return credits.** Every packet RAPIDS consumes returns its full credit, including one it discards (bad header, bad tag, disabled channel). No path consumes a packet silently. - **Stalled data is a monitor error, both directions.** RAPIDS carries ocelus checks, each a threshold in cycles (10 us) that flags an error on MonBus: - Sink: data present in the sink SRAM that has not moved. - Source: a source channel expecting data (reads outstanding) whose SRAM has received none. - **Return credits promptly.** RAPIDS may coalesce the returns already pending when it sends one, but never holds credits back to build a batch while the sender may be waiting for them. - **Straps are a Hive-level contract.** The Hive sets every strap and parameter (`TILE_LAND_FLITS`, the descriptor credits, the cells' and cores' straps) and checks they agree; it is the final arbiter. A block built on its own (one tetrad, one RAPIDS) is only as consistent as the parameters it is given. - **Resync and reset only when quiescent.** A global quiescent signal, covering every cell and tile, gates any resync and any reset or partial reset (§9, §11.10), so no packet is in flight against credits being restored or cleared.

5.11.10 11.10 Credit advertisement

Each side advertises its own credits in-band. Nothing is loaded from CSRs, so the counts cannot drift from the hardware.

Credit packet operations

op in a `credit` packet	Meaning	Used by
0 = add	Add N to the counter	Credit returns, both directions
1 = set	Replace the counter with	Advertisements at boot,

op in a credit packet	Meaning	Used by
	N	resync

Boot: 1. After reset every cell and control core sends one `set` credit packet to `level` 3, under its external ID, carrying its full input-buffer size in flits (owner direction 2026-10-18: the cell or core originates it; a delta node never does). RAPIDS's external counters start at 0, so RAPIDS cannot send to a cell or core that has not advertised. delta nodes do not advertise: they are legal by topology. 2. The advertisements ride delta's hardwired reset-default route toward the RAPIDS boundary (delta HAS ch07 §3), so no routing table is needed first. 3. They need no strap credit: credit packets travel on their own interface and the sink port always accepts them. 4. The advertisement doubles as the cell's or core's "alive": RAPIDS accepts strap traffic only from a sender that advertised in the window. 5. **Legal destinations** (owner direction 2026-10-18). RAPIDS opens an advertisement window when it leaves reset. When the window closes, every node's internal ID is legal, and every external counter still at 0 marks an **illegal** destination: nothing there advertised, whether an absent cell, an absent core or a spare index. RAPIDS saves the result as a 64-bit legal-destination mask, one bit per counter. - A descriptor whose destination is illegal is refused as an invalid descriptor, not sent and not left waiting for credits that will never come. - Only the window decides legality. A legal destination whose counter later reaches 0 because its credits are spent is waiting, not illegal. - The mask is readable by software (a CSR) and stays as saved until the next reset.

RAPIDS's side: straps, not advertisements. RAPIDS never advertises its free sink space as a total. If it broadcast "channel 3 has 512 flits free", every cell would believe it could send 512 flits, and the sink would be overcommitted many times over. Its space toward each sender is a fixed strap instead (§11.9), reserved by construction.

Resync after root failover: credit packets in flight in the failed root are lost. Without a resync the lost credits leak until the counters reach zero and traffic stops. A resync runs only while the global quiescent signal is asserted (§11.9, hazard rules): with nothing in flight, a restored strap is never counted twice against data sent under the old one. 1. After the mux switches and the tile is quiescent, RAPIDS sends every cell and core a `set` credit packet, as one multicast, restoring its strap counter to what RAPIDS can now accept from it. 2. Each cell and core answers with a `set` advertisement of its current free input-buffer space. 3. RAPIDS rebuilds its counters from the answers. The saved legal-destination mask stands: a resync does not reopen the window, so it never waits on a destination that was illegal at boot.

A destination that leaves reset on its own uses the same path: its fresh advertisement replaces RAPIDS's view of it.

5.11.11 11.11 Cell-issued descriptors

A tetrad's control core (gatling) can make RAPIDS read memory and send the data to a cell by handing RAPIDS a complete source descriptor (owner direction 2026-10-18: gatling creates the descriptors). It is a pull: the core asks, the RAPIDS source half does the read and the send.

1. The core sends a `desc` packet to `level` 3 through RAPIDS's sink port: the header (tag = the source channel to run it on, length = 32) followed by one 256-bit RAPIDS source descriptor in the ordinary format. `src_addr` is the memory to read; `dst_addr` carries the destination coordinate and target address (RAPIDS HAS, Descriptor Format); `ptype`, `op`, `length` and the chaining fields mean what they always mean.
2. The sink port recognises `desc`, never stores it in a sink channel's SRAM, and passes the descriptor to the source half, which writes it straight into the tagged channel's descriptor queue, with no fetch.
3. An idle channel starts on it, as if kicked with the descriptor already fetched. A busy channel runs it after the descriptors already queued. A non-zero next pointer chains on into memory-resident descriptors as usual.
4. The source half reads `src_addr` and sends the data, with a header built from the descriptor, out of the source port to the destination it names.

Flow control (owner 2026-10-18): there is one descriptor queue, feeding the schedulers. `desc` packets land in an intake FIFO as deep as the descriptor credits handed out (a parameter); the intake FIFO is muxed into the put side of the descriptor queue, beside the memory fetch path. A core sends a `desc` packet only against its strapped share of those credits (§11.9, credit class 1), and RAPIDS returns an entry when the descriptor moves into the descriptor queue.

5.11.12 11.12 RAPIDS implementation status (2026-10-17)

Built in the byte-granular RAPIDS behind `TILE_HDR` = 1 (default 0; needs `SHARED_ENGINES` = 1 and `DATA_WIDTH` >= 256). RAPIDS Beats does not have it.

Item	State
Header packing (§11.5), payload types (§11.2)	Built: `rapids_pkg::tile_hdr_pack`. The header is the first beat of every packet on both ports, bits [127:0]; on an AXIS plane it is that first beat too, not `TUSER`
Source header from the descriptor (§11.7)	Built. Tag = the channel, source coordinate = the `TILE_SRC_COORD` parameter (level 3). A data packet's header leaves only when its first data beat is ready, so the two go back to back. A source descriptor with a `ptype` other than `cfg`/`prog`/`mem` is refused as an invalid descriptor

Item	State
Read request (op = 1)	Built: a header-only packet, no memory read. As for a write, the descriptor completes when its packet is queued, not when the header has left; a zero-length request sends nothing
Sink header decode	Built (rapids_tile_hdr_rx): cfg/prog/mem writes are stripped and delivered by stream ID; credit, read requests into RAPIDS and reserved types are consumed, never stalled, and counted or flagged
Sink stream = tag	On RAPIDS's sink link the stream ID (CXSSRCID, the channel) must equal the header tag: the root sets it from the tag. RAPIDS delivers by stream ID and flags a mismatch (sts_tile_tag_mismatch)
Cell-issued descriptors (§11.11)	Built, with a one-descriptor slot per source channel instead of descriptor grants: a desc with no free slot, one arriving during a channel reset, or an EXT one is dropped and sets that channel's sticky sts_tile_desc_dropped bit. The channel and the work it holds carry on
End-to-end credits (§11.9, §11.10)	Built behind TILE_CREDITS = 1 (2026-10-18): 64 destination counters in the source half, the internal and external ID of every node (rapids_tile_credits; index map §11.9), external counters filled by class-0 set / add credit packets from cells and cores; a packet to a cell or core leaves only when its counter covers every flit, header included.

Item	State
	Node internal IDs are legal by topology, never held or charged, and their credit packets are ignored; a read request to a node is refused. The advertisement window (SRC.TILE_ADV_WINDOW, cycles from reset) closes into the saved 64-bit legal-destination mask (SRC.TILE_LEGAL_DEST_LO / _HI); tile descriptors wait for it and one to an illegal destination is refused as invalid. A channel reset does not refund credits already charged
Straps toward RAPIDS (§11.9, §11.11)	Not built: no per-cell landing space (TILE_LAND_FLITS), no desc intake queue, no credit returns from RAPIDS; RAPIDS ignores class-1 credits (rapids TASK-027)
Status	CSRs: SNK.TILE_STATUS and SRC.TILE_DESC_DROPPED (W1C), SNK.TILE_COUNTS, and for credits SRC.TILE_LEGAL_DEST_LO / _HI / SRC.TILE_CREDIT_STATUS; the same values on the top-level sts_tile_* ports

5.11.13 11.13 Tetrad control core (owner direction 2026-10-17; deferred 2026-10-18; ruling 2026-10-19)

Deferral (owner direction 2026-10-18): the tetrad engine is defined **after the cell is fully defined and working** — the cell is the lowest module (cell PRD v0.3 §1). The owner-authored gatling-rv32itetrad barrel-core PRD (2026-10-17) is the leading candidate, with kestrel-class remaining a candidate flavor via CORE_FLAVOR.

Owner ruling 2026-10-19: gatling sequences the four **alu-infer** ALUs of its adjacent cells. Each barrel context's POP presents operands to the cell ALU's issue handshake, and PUSH retires result lines; the cell's slice-3 exec FSM remains the standalone bring-up driver with the same client contract. Gatling never originates config traffic.

Each tetrad carries **one RV32I control core** — kestrel-class, `CORE_FLAVOR`-swappable, defined with its full duty list in alu-infer PRD §5. The model, recorded here so the tile records it:

- **Attach.** The core sits at its tetrad node's **local port**, the same relationship the cell has with its own node. It is not a routing level, carries no coordinate of its own, and is reached as its tetrad: `prog` and `cfg` packets addressed `{level = 1, tetrad X/Y}` (§11.3) exit the tetrad node's local port into the core. Its firmware load rides the existing `prog` payload type (§11.2) — in-band like everything else.
- **In-band everything, no config origination.** All data, config, and programming for the tile blocks arrive with the delta traffic (owner ruling 2026-10-17). The core does not originate config traffic: the cells' programming (`FUNC_SEL`, `OP_LINES`, …) arrives in-band addressed to each cell (`level = 0`), carried by the stream. The core is a **local reaction agent**: it reads its four cells' status and IRQs over the local AXIL register plane, runs decision firmware (layer-boundary detection, flush policy), and exercises *operational* registers only (flush-and-send, soft reset, IRQ clear — the AXIL-for-registers plane, not fabric config).
- **Custom instructions stay minimal** (a custom-0 `KICK`/`WAIT` pair for local coordination, provisional pending the tetrad MAS); the `FLUSH` instruction is part of result-flush (§11.14). Encoding lands in the cell/tetrad MAS (§10).
- **Observability is unchanged.** SERV and the MonBus aggregation (§5) remain the tile's observability plane; the core reads registers and reacts — it takes no MonBus tap by default.
- **Why per-tetrad.** Four independent control points per tile (one per tetrad) allow layer pipelining across tetrads and one group-done per four cells, without sixteen instruction streams. A tetrad steps as 4 cells × 2 bf16 lanes = 8 bf16 lanes.

5.11.14 11.14 Result flush to the tile I/O (owner direction 2026-10-18; defined 2026-10-19)

Delta **must** be able to carry a cell's result data to `level` 3, the tile I/O, and so into RAPIDS. It is the path computed results leave the tile by:

1. **Detect.** Gatling, the tetrad engine (§11.13), detects a flush-to-memory condition. When to flush is program policy, not fixed hardware.
2. **Command.** Gatling sends a flush command to the result drain controller in each cell it applies to. **Gatling tells the cell the destination** (owner

direction 2026-10-18): the cell does not choose it and does not need to know where RAPIDS attaches.

3. **Drain.** The controller pushes the cell's result memory out through the packetizer: write-format packets (`op` = 0) to `{ext 0, level 3}`, source = the cell's external ID `{1, 0, X, Y, Z}`, tag = the RAPIDS sink channel, each sent against the cell's `TILE_LAND_FLITS` strap (§11.9).
4. **Carry and land.** Delta routes each packet to the tile I/O. RAPIDS's sink strips the header and delivers the data by tag into that sink channel, which writes it to memory.

Flush instruction (owner ruling 2026-10-19): gatling PRD v0.2 adds the `Xcell FLUSH` custom instruction: `funct3 = 011`, `rs1[14:0]` = drain destination coordinate, `rs1[18:15]` = sink-channel tag. The instruction programs the cell's `FLUSH_DEST` / `FLUSH_TAG` and triggers the cell's flush-and-send. The coordination pair `KICK` (`funct3 = 100`) / `WAIT` (`funct3 = 101`) is also added (provisional pending the tetrad MAS). The engine is still deferred until the cell works (§11.13); the instruction encoding and cell CSR details land in the cell/tetrad MAS.

Link	State
Gatling detects and commands	Defined in gatling PRD v0.2: `Xcell FLUSH`, `KICK`, `WAIT`; engine deferred until the cell works (§11.13)
Cell drain controller	Defined: the cell PRD has the packetizer and `CTRL` flush-and-send; no cell RTL beyond alu-infer
Delta to `level` 3	Interim: delta has no `level` field, so the destination gatling hands the cell includes the attach cell's delta coordinate, and the cell writes it into delta's header; delta needs no change for that. Permanent: `level` and `ext` in delta's header (delta DELTA-011)
RAPIDS sink delivery by tag	Built (§11.12)
Straps toward RAPIDS	Not built (rapids TASK-027). Without the reserved landing space a drain into a full sink channel stalls in the routers

5.11.15 11.15 Multi-path routing and failover: open hazards (2026-10-18)

Resilience is to come from delta route tables that hold a primary and an alternate next hop, with failover as an in-band table write and no new wires (delta DELTA-013). The tile topology already has the alternates: each tetrad node reaches the other three directly (K4) or through one of them, and every tetrad node links to both roots. Then a packet's path depends on table state, so a request and its reply can take different paths. The hazards, worked in DELTA-013, are recorded where they bite:

- **Delta** (delta HAS ch07, Resilient Routing: Open Hazards): mixed tables during a node-by-node update; the VC0 escape channel losing connectivity when a failed link sits on it, so the escape rule (up/down over this hierarchy is the natural one) is re-derived on every failover; the config route crossing the failure; and a misroute drop losing a `credit` or data packet, which breaks "always return credits" (§11.9).
- **Ordering is gone between packets.** Only the flits of one packet stay in order. Order becomes a requester-side property: a transaction ID per request, echoed in the reply. RAPIDS's cases (read replies sharing a sink-channel tag, a resync `set` racing an older `add`, a write followed by a packet that depends on it) are in the RAPIDS MAS, ch03 Tile Packet Header, Ordering Across the Fabric. Nothing in the tile may be built on fabric order.
- **Design fork:** fully adaptive on both legs, or pinned forward (requests follow the table's path, deterministic per table state) with an adaptive return. Pinned forward is the leading option, being much easier to verify and enough for surviving a forward-path failure; it still reorders at a failover flip and on the return leg.

5.12 12. Milestones

Owner direction 2026-10-19: the tile work is staged so that the cell and tetrad are proven before rapids and real data are brought in.

Milestone	What is proven
M1	A fully working **tetrad** on the FPGA — 4 cells + gatling sequencer over delta; proves the cell datapath, credit loop, and programmability.
M2	A **tile** running in isolation (with injectors) on an FPGA — 16 cells, tetrads, roots, and MonBus levels; no rapids.
M3	**Rapids** + **tile** working artificially (injectors/test data) — full source/sink loop with tile headers, end-to-end credits, and straps.
M4	**Rapids** + **tile** working on real data and benchmarks.

5.13 References

- `docs/superpowers/specs/2026-10-13-delta-mesh-design.md` — working design record (§2 architecture, §5.4 mode catalog).
- `3d-mesh-tile.drawio` — owner's K4-at-both-scales figure.
- `delta_has/ch07_mesh_network/` — the node/network chapters in delta's HAS (cell/flit, routing, flow control, QoS, multicast, modes, observability, verification).
- `noc-ip/delta_spec` — the aspirational NoC draft this subsystem grew out of.

Last Updated: 2026-10-19 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

6 KESTREL-RV32I Micro-Architecture Specification Index

Version: 0.2 **Date:** 2026-10-20 **Purpose:** Block-level micro-architecture specification for the kestrel-rv32i single-cycle RV32I core

---

6.1 Document Organization

Note: All chapters linked below for automated document generation.

6.1.1 Front Matter

- Document Information

6.1.2 Chapter 1: Overview

- Design Philosophy and Machine Organization

6.1.3 Chapter 2: Functional Blocks

- kestrel_core (top)
- kestrel_decode
- kestrel_imm_gen
- kestrel_alu
- kestrel_regfile
- kestrel_mem_loader

6.1.4 Chapter 3: Internal Interfaces

- Package Types and Top-Level Wiring

6.1.5 Chapter 4: Micro-Architecture Behavior

- The Single Cycle, Step by Step
- Cross-Word Misaligned Retry
- Halt, IALIGN, and the System Layer
- RVFI Aggregation

6.1.6 Chapter 5: Verification Hooks

- Verification Hooks and Observability

6.1.7 Chapter 6: Configuration

- Parameters, Defines, and Filelists

6.2 Related Documentation

- **KESTREL HAS** - Hardware Architecture Specification (external view)
- **Simplified RV32I: kestrel** - Tutorial-study book this core is documented by
- **RISC-V Falcon Suite README** - Suite overview and rung ladder
- **riscv-formal rvfi.md** - Retire-channel field semantics
- **Testplans** - V&V cross-reference (six plans)

Last Updated: 2026-10-07 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

7 rtl-math module index

RTL: `rtl/math/` (174 modules) **Tests:** `val/math/` **Common building blocks:** rtl-common

The complete `math_*` arithmetic library: adders, subtractors, multipliers, compressors, prefix cells, and the BF16 / FP8 / FP16 / FP32 / IEEE-754 floating-point modules — every family, with the files that make it up.

7.1 Overview

This page is the catalogue, not the guide. Read the **Math Library overview** first: it organizes the whole `math_*` family by operation → methodology — Brent-Kung / Han-Carlson adders, Dadda / Wallace multipliers, IEEE-754 fp32/fp16/bf16/fp8 operators, conversions, and activations — each with its research reference and a link to the detailed doc below, plus the `bin/` code-generation automation. Come back here when you need the exact file list. That's the one thing this page does better than any other.

7.2 Module categories

7.2.1 Basic arithmetic

- **math_adder_basic** — single-bit adders (full and half adder)
  - Includes: `math_adder_full.sv` (dedicated page), `math_adder_half.sv`, `math_adder_full_nbit.sv`
- **math_adder_ripple_carry** — multi-bit ripple carry adder
- **math_adder_pg_chain** — fast carry lookahead adder
- **math_adder_carry_save** — carry-save adder for multiple operands
  - Includes: `math_adder_carry_save.sv`, `math_adder_carry_save_nbit.sv`

7.2.2 Advanced adders

- **math_adder_brent_kung** — Brent-Kung parallel prefix adder family (8/16/32/64-bit)
  - Includes: `math_adder_brent_kung_008.sv`, `math_adder_brent_kung_016.sv`, `math_adder_brent_kung_032.sv`, `math_adder_brent_kung_064.sv`
  - Sub-modules: `math_adder_brent_kung_pg.sv`, `math_adder_brent_kung_black.sv`, `math_adder_brent_kung_gray.sv`, `math_adder_brent_kung_bitwisepg.sv`, `math_adder_brent_kung_grouppg_*.sv`, `math_adder_brent_kung_sum.sv`
- **math_adder_han_carlson** — Han-Carlson hybrid parallel prefix adder family (16/22/32/44/48/72-bit)
  - Includes: `math_adder_han_carlson_016.sv`, `math_adder_han_carlson_022.sv`, `math_adder_han_carlson_032.sv`, `math_adder_han_carlson_044.sv`, `math_adder_han_carlson_048.sv`, `math_adder_han_carlson_072.sv`
  - Building blocks: `math_prefix_cell.sv`, `math_prefix_cell_gray.sv`
- **math_adder_hybrid_ripple_prefix** — hybrid adder: low-order ripple chain + single Kogge-Stone tree for the upper bits (8/16/32/64-bit)
  - Includes: `math_adder_hybrid_ripple_prefix.sv` (parameterized core), `math_adder_hybrid_ripple_prefix_008.sv`, `math_adder_hybrid_ripple_prefix_016.sv`, `math_adder_hybrid_ripple_prefix_032.sv`, `math_adder_hybrid_ripple_prefix_064.sv`

- **math_addsub** — combined adder/subtractor
  - Includes: `math_addsub_full_nbit.sv`

7.2.3 Division

- **math_dividers** — integer divider family: restoring, non-restoring, and SRT radix-2 / radix-2-carry-save / radix-4 (parameterized `DATA_WIDTH`)
  - Includes: `math_divider_restoring.sv`, `math_divider_nonrestoring.sv`, `math_divider_srt_radix2.sv`, `math_divider_srt_radix2_csa.sv`, `math_divider_srt_radix2_csa_structural.sv` (cell-built A/B twin), `math_divider_srt_radix4.sv`

7.2.4 Subtraction

- **math_subtractor** — subtractor family (single-bit and multi-bit)
  - Includes: `math_subtractor_full.sv`, `math_subtractor_half.sv`, `math_subtractor_full_nbit.sv`, `math_subtractor_ripple_carry.sv`, `math_subtractor_carry_lookahead.sv`

7.2.5 Multiplication

- **math_multiplier_wallace_tree** — Wallace tree multiplier family (8/16/32-bit)
  - Includes: `math_multiplier_wallace_tree_008.sv`, `math_multiplier_wallace_tree_016.sv`, `math_multiplier_wallace_tree_032.sv`
  - CSA variants: `math_multiplier_wallace_tree_csa_008.sv`, `math_multiplier_wallace_tree_csa_016.sv`, `math_multiplier_wallace_tree_csa_032.sv`
- **math_multiplier_dadda_tree** — Dadda tree multiplier family (8/16/32-bit)
  - Includes: `math_multiplier_dadda_tree_008.sv`, `math_multiplier_dadda_tree_016.sv`, `math_multiplier_dadda_tree_032.sv`
- **math_multiplier_dadda_4to2** — Dadda tree multiplier with 4:2 compressors (8/11/24-bit)
  - Includes: `math_multiplier_dadda_4to2_008.sv` (BF16), `math_multiplier_dadda_4to2_011.sv` (FP16), `math_multiplier_dadda_4to2_024.sv` (FP32)
  - Building blocks: `math_compressor_4to2.sv`

- **math_multiplier_basic** — basic multiplier components
  - Includes: `math_multiplier_basic_cell.sv`, `math_multiplier_carry_save.sv`

7.2.6 BF16 floating-point arithmetic

- **math_bf16_adder** — pipelined BF16 adder with configurable latency
  - Includes: `math_bf16_adder.sv`
  - Dependencies: `shifter_barrel.sv`, `count_leading_zeros.sv`
- **math_bf16_multiplier** — complete BF16 multiplier with IEEE 754 compliance
  - Includes: `math_bf16_multiplier.sv`
  - Sub-modules: `math_bf16_mantissa_mult.sv`, `math_bf16_exponent_adder.sv`
- **math_bf16_mantissa_mult** — BF16 mantissa multiplier with normalization detection
  - Includes: `math_bf16_mantissa_mult.sv`
- **math_bf16_exponent_adder** — BF16 exponent computation with overflow/underflow detection
  - Includes: `math_bf16_exponent_adder.sv`
- **math_bf16_fma** — BF16 Fused Multiply-Add with FP32 accumulator for AI training
  - Includes: `math_bf16_fma.sv`
- **math_bf16_extended** — extended BF16 modules (29 modules)
  - Activation functions: `relu`, `leaky_relu`, `gelu`, `sigmoid`, `tanh`, `silu`, `softmax_8`
  - Math operations: `exp2`, `log2`, `log2_scale`, `reciprocal`, `fast_reciprocal`, `divider`, `goldschmidt_div`, `newton_raphson_recip`
  - Comparison/Selection: `comparator`, `clamp`, `min`, `max`, `min_tree_8`, `max_tree`, `max_tree_8`
  - Format conversions: `to_fp16`, `to_fp32`, `to_fp8_e4m3`, `to_fp8_e5m2`, `to_int`, `int_to_bf16`, `scale_to_int8`

7.2.7 FP16 (half-precision) floating-point

- **math_fp16_modules** — FP16 module collection (17 modules)
  - Activation functions: `relu`, `leaky_relu`, `gelu`, `sigmoid`, `tanh`, `silu`, `softmax_8`

- Comparison/Selection: `comparator`, `clamp`, `min`, `max`, `min_tree_8`, `max_tree_8`
- Format conversions: `to_bf16`, `to_fp32`, `to_fp8_e4m3`, `to_fp8_e5m2`

7.2.8 FP32 (single-precision) floating-point

- **math_fp32_modules** — FP32 module collection (17 modules)
  - Activation functions: `relu`, `leaky_relu`, `gelu`, `sigmoid`, `tanh`, `silu`, `softmax_8`
  - Comparison/Selection: `comparator`, `clamp`, `min`, `max`, `min_tree_8`, `max_tree_8`
  - Format conversions: `to_bf16`, `to_fp16`, `to_fp8_e4m3`, `to_fp8_e5m2`

7.2.9 FP8 (8-bit floating-point)

- **math_fp8_modules** — FP8 E4M3 and E5M2 module collection (44 modules)
  - E4M3 (inference): `adder`, `multiplier`, `fma`, activations, comparisons, conversions
  - E5M2 (training): `adder`, `multiplier`, `fma`, activations, comparisons, conversions
  - Format conversions between E4M3, E5M2, BF16, FP16, FP32

7.2.10 IEEE 754-2008 compliant arithmetic

- **math_ieee754_modules** — IEEE 754-2008 arithmetic (12 modules; FTZ by default, full gradual underflow via `SUBNORMAL_SUPPORT=1` on every block; all eight arithmetic units run exact-integer-oracle suites at both parameter values, and the divider/sqrt TBs pin the FSM latencies exactly)
  - FP16: `adder`, `multiplier`, `fma`, `mantissa_mult`, `exponent_adder`
  - FP32: `adder`, `multiplier`, `fma`, `divider`, `sqrt`, `mantissa_mult`, `exponent_adder`
  - Features: `SUBNORMAL_SUPPORT=0` FTZ (legacy default) / `=1` IEEE 754-2008 gradual underflow, complete special-case handling, full status flags

7.2.11 Compressors and prefix cells

- **math_compressor_4to2** — 4:2 compressor for fast parallel reduction
- **math_mod_3_compress** — combinational `X - (X mod 3)` carry-save rounding (monbus record packing)
- **math_prefix_cell** — black cell for parallel prefix adders
  - Includes: `math_prefix_cell.sv`

- **math_prefix_cell_gray** — gray cell for parallel prefix adders (area-optimized)
    - Includes: `math_prefix_cell_gray.sv`

7.3 Related Modules

- rtl-common index — counters, FIFOs, arbiters, CDC, data integrity

7.4 Navigation

- Documentation index

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

8 Document Information

8.1 Overview

8.1.1 ALU-INFER Micro-Architecture Specification

Document information

Property	Value
Document Title	ALU-INFER Micro-Architecture Specification
Version	0.50 (pinned per owner, “for now”)
Date	2026-10-17
Status	Draft — v1 RTL lives in `alu-infer/rtl/` (landed 2026-10-17/18 in `cell/rtl/`, func+gate green; moved to the alu-infer component 2026-10-20): `alu_infer_top.sv`, `alu_infer_rows.sv`, `alu_infer_pipe.sv`, `alu_infer_acc.sv`. The MAS’s planned submodules `alu_infer_decompose/select/compose/decode` are folded into `alu_infer_rows.sv` in the as-built RTL. The tag-based alignment of the owner ruling 2026-10-18 is the as-built realization since 2026-10-19: `alu_infer_pipe` is a tag FIFO with an arrival join; parameters are `PIPE_DEPTH` (default 8, range [2,16]), `ADDER_LATENCY` (default 3, range [0,4]), and `TAG_W` (8, per ruling). Chapters 2–4 are reconciled to the as-built code and carry the open decision IDs M1–M8.
Classification	Open Source - MIT License

8.1.2 Document Purpose

This Micro-Architecture Specification (MAS) documents how the cell's inference ALU is built:

- The `alu_infer_top` module tree and its parameters
- The seven function rows, each with the `rtl/math` block it instantiates and its wiring decisions
- The fp32 MAC accumulator — the block's only state — and its CLEAR / ACCUMULATE / DRAIN semantics
- The signal-level ALU↔FSM interface and its valid/ready elastic-pipeline handshake
- Cycle-level behavior per opcode class, with timing diagrams
- Verification hooks: the golden model, the built-in library cross-check, and the coverage matrix
- The configuration surface: encodings, parameters, and the open-items register

Yes, the "(planned)" markers of earlier drafts get called out up front rather than left for you to discover. The v1 RTL lives in `alu-infer/rtl/` (landed in `cell/rtl/` on 2026-10-17/18, func+gate green; moved to the alu-infer component on 2026-10-20): `alu_infer_top.sv`, `alu_infer_rows.sv`, `alu_infer_pipe.sv`, `alu_infer_acc.sv`. The MAS's planned submodules `alu_infer_decompose/select/compose/decode` are folded into `alu_infer_rows.sv` in the as-built RTL. This book now documents the as-built implementation that the cell's RTL is checked against — including the tag-based alignment of the owner ruling 2026-10-18, implemented 2026-10-19: `alu_infer_pipe` is a tag FIFO with an arrival join (Chapter 4), and the retired alignment-timer mailbox's `ROW_LATENCY` parameter is replaced by `PIPE_DEPTH` and `ADDER_LATENCY`. For what the ALU must do and why, see the companion **alu-infer PRD** (normative for the ALU block); for the cell around it, see the **cell PRD**.

8.1.3 Intended Audience

- RTL designers implementing the ALU inside the cell's `rtl/`
- Verification engineers developing the cell's testbenches
- Design reviewers checking the future implementation against intent
- Students reading the bf16 inference datapath top to bottom

8.2 Design Notes

8.2.1 Revision History

Revision history

Version	Date	Author	Description
0.50	2026-10-18	RTL Design Sherpa	Revs pinned at 0.50 per owner ("for now," while the ALU heads to the board). Content: tag-based elastic alignment ruling (PRD v0.50) — 8-bit tag per loaded invocation, common to all ops that must run in parallel; FIFO-tracked; op valid asserts only when all valids of its tag group are ready. **Implemented 2026-10-19** (func+gate green, 13 cocotb tests × {gate,func} × PIPE_DEPTH {4,8}): `alu_infer_pipe` is now the tag FIFO with an arrival join; results retire in issue order at natural per-row latency (combinational

Version	Date	Author	Description
			rows the cycle after issue, add/sub `ADDER_LATENCY` cycles after), a stalled consumer no longer blocks issue until the FIFO fills, and `ROW_LATENCY` is replaced by `PIPE_DEPTH` (default 8, [2,16]) and `ADDER_LATENCY` (default 3, [0,4]). The PRD's "exactly `ROW_LATENCY` cycles" phrasing described the retired mailbox realization; the seam contract (two valid/ready pairs, in-order, II=1, no tags on the seam) is unchanged. Carries Rev 0.5's port-width correction.
0.5	2026-10-17	RTL Design Sherpa	Port-width correction (PRD v0.9): operand ports stay `LINE_W` each — a

Version	Date	Author	Description
			binary/mac invocation pops one line per side (OP_LINES counts the A+B pair); the v0.7 widening was a line/lane mis-count. Same-clock capture ruling unchanged.
0.4	2026-10-17	RTL Design Sherpa	Owner accept-and-defer pass (PRD v0.8): M7 and the `acc_ctrl` encoding accepted as drafted (“works for now”); per-row latency minimization deferred to FPGA bring-up (M8); PRD §8 result-flag question closed (packed low, one bit per lane).
0.3	2026-10-17	RTL Design Sherpa	Owner rulings closing the implementation gaps (PRD v0.7): `OP_LINES × LINE_W` operand ports with same-clock capture; scheduler-paced feedback (the ALU makes no

Version	Date	Author	Description
			autonomous pacing decisions); CLEAR and errored opcodes retire their slots; `result_flags` = one mask bit per lane, packed low (M5 closed).
0.2	2026-10-17	RTL Design Sherpa	Elastic-pipeline interface (owner direction, PRD v0.6): the ALU↔FSM seam is a valid/ready elastic pipeline — `valid`/`ready` issue, `result_valid`/`result_ready` retire, backpressure freezes the alignment timer; Waveform 3.1 redrawn with backpressure at both ends. Carries Rev 0.1's decisions.
0.1	2026-10-17	RTL Design Sherpa	Initial MAS release, drafted with the PRD v0.3 decisions: bf16 element model on the `math_bf16_*` library, fp32 MAC accumulation

Version	Date	Author	Description
			(presentation semantics decided as M7), per-tetrad kestrel-class control interface, in-band config/programming.

8.3 References

8.3.1 Related Documents

- **alu-infer PRD** (../../../PRD.md) — product definition, normative for the ALU block
- **cell PRD** (../../../../cell/docs/PRD.md) — the cell around the ALU: memories, execution FSM, packetizer, credit logic, §7 register map
- **TILE_SPEC §11.13** (../../../../tile/docs/TILE_SPEC.md) — the tetrad control core's place in the tile
- **KESTREL-RV32I MAS** (../../../../../riscv-ip/kestrel-rv32i/docs/kestrel_mas/kestrel_mas_index.md) — the tetrad control core's reference core
- **rtl-math book** (../../../../../../../docs/markdown/rtl-math/index.md) — the library the function rows instantiate
- **rtl-math references index** (../../../../../../../rtl/math/references/README.md) — the math-paper archive (bf16 format, activations, quantization)

Last Updated: 2026-10-19

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

9 Design Philosophy and Datapath Organization

9.1 What This Block Is

`alu_infer` is the cell's compute engine: a bf16 function array that the cell's execution FSM drives with a simple pop → compute → push loop. It is deliberately *not* a processor — no fetch, no program counter, no branches. Its programmability lives one level up: a per-tetrad RV32I control core (gatling-class barrel RISC-V) reacts to the cells' status, and the tile's data/config/programming stream arrives in-band on delta. The ALU's only job is to turn operand lines into result lines, one config-selected function at a time.

The v1 RTL lives in `alu-infer/rtl/` (landed 2026-10-17/18, func+gate green; moved from `cell/rtl/` to the alu-infer component on 2026-10-20): `alu_infer_top.sv`, `alu_infer_rows.sv`, `alu_infer_pipe.sv`, `alu_infer_acc.sv`. The planned submodules `alu_infer_decompose`/`select`/`compose`/`decode` of earlier MAS drafts are folded into `alu_infer_rows.sv` in the as-built code. The alignment mechanism is the owner-ruled tag FIFO (2026-10-18, implemented 2026-10-19): each issue takes an 8-bit tag, `alu_infer_pipe` tracks the tags in flight, and a result is offered only when every op of its tag group has arrived — so quick rows compute at their natural speed and align at the join instead of carrying padded pipe stages.

Three commitments shape every decision in this specification.

1. The math library is the datapath. Nearly every function row instantiates an existing `rtl/math` block — `math_bf16_adder`, `math_bf16_multiplier`, `math_bf16_fma`, `math_bf16_comparator`, `math_bf16_min`, `math_bf16_max`, `math_bf16_relu`. The ALU therefore inherits three things for free: structure already documented in the rtl-math book, verification already done by the library's own dv, and convergent-thinking citations already archived in `rtl/math/references/`. New arithmetic is added to the library first and consumed here second; rows without a block (copy, mask packing) are wires and glue. This is the same consumption pattern as the falcon suite's garuda rung, which takes the library's IEEE-fp32 blocks behind its reservation stations.

2. FMA-first, fp32 accumulate. Inference time is dominated by multiply-accumulate, so the mac row is the workhorse: `math_bf16_fma` computes the bf16 product exactly in fp32 (8×8 significand bits fit fp32's 24) and adds it into a per-lane fp32 accumulator — one rounding per operation, one rounding to bf16 at result presentation. Every other row is sized to sit beside the FMA without dwarfing it.

3. A function table, not a function zoo. Opcodes 0–7 are RTL table rows — the select is a mode mux driven by `func_sel`, never an arbiter, and growth rows (activations, reciprocal/divide, conversions) are added as rows without disturbing the handshake, the lanes, or the verification scaffolding.

9.2 The Dataflow in One Paragraph

Both operand lines — A and B, one per side — arrive **in the same clock** from the execution FSM's input-memory pops. The element decompose block splits each 32-bit line into two bf16 lanes. All seven function rows see the same lanes; the row named by `func_sel.opcode` computes, and the result select forwards that row's two lanes to the line composer, which packs them back into a 32-bit result line. The compare row additionally emits a per-lane mask as `result_flags`. State exists in exactly one place — the fp32 MAC accumulator, private to opcode 4, managed by `acc_ctrl` — so every other row is a pure function of its operands, which is what makes the verification story (Chapter 5) tractable.

9.2.1 Figure 1.1: ALU-INFER datapath overview (definition level)

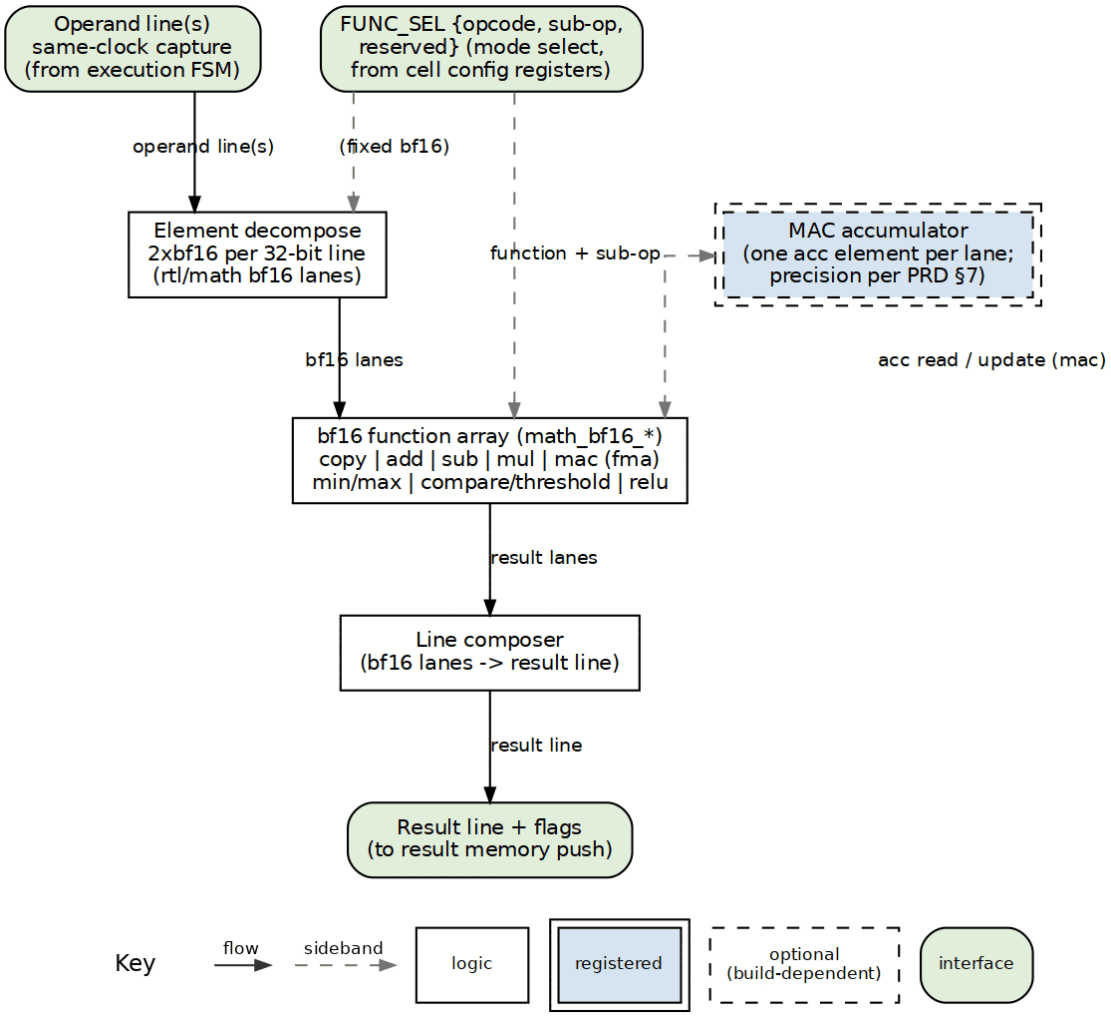


Figure 1.1: ALU-INFER datapath overview

The figure above is the context view — operand flow down the center, `FUNC_SEL` and the MAC accumulator dashed as sideband. The full block diagram with every row and its library block is Figure 2.1 in the next chapter.

9.3 Relationship to the Cell and the Tetrad Core

The ALU speaks only to the cell's execution FSM (Chapter 3): operands, result, handshake, and two config inputs (`func_sel`, `acc_ctrl`). The FSM owns sequencing, the memories, and the credit loop. The tetrad control core — one gatling-class barrel RV32I per tetrad — is out of this block entirely: it reads cell status and IRQs over the AXIL register plane and exercises operational registers; per the owner ruling of 2026-10-17 it originates no config traffic, and cell *programming* (including this ALU's `FUNC_SEL` register) arrives in-band with the delta traffic. Observability is the Ocelus MonBus plane's job, not the ALU's; this block exposes no monitor taps by default (open item M2 asks whether the library blocks' IEEE exception outputs should surface as `result_flags`).

9.4 Status of This Specification

The v1 RTL is built and green. Chapters 2–4 document the as-built implementation in `alu-infer/rtl/` (`alu_infer_top.sv`, `alu_infer_rows.sv`, `alu_infer_pipe.sv`, `alu_infer_acc.sv`, landed 2026-10-17/18, func+gate green). Where the PRD left a choice open, this specification either decides it (and records the decision) or carries it as an open item M1–M8 in Chapter 6. The tag-based alignment mechanism of Chapter 4 is the as-built realization since 2026-10-19; it superseded the alignment-timer mailbox of the first v1 build without any interface change — the PRD's "presented exactly `ROW_LATENCY` cycles after issue" phrasing described that mailbox and is superseded with it (the seam contract it sat under — two valid/ready pairs, in-order retirement, II=1 — is unchanged).

Last Updated: 2026-10-19

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

10 alu_infer_top (module tree)

10.1 Purpose

`alu_infer_top` is the ALU block instantiated inside the cell, at the seam the cell PRD calls “ALU + execution FSM”. It owns everything between the operand-line inputs and the result-line outputs: element decompose, the seven function rows, the fp32 MAC accumulator, the result select, and the line composer.

10.2 Module Tree

```
alu_infer_top                                  — the ALU block at the
cell seam
├── alu_infer_rows                             — the function table
│   ├── row_copy                               — bypass (wires)
│   ├── row_addsub    (math_bf16_adder)        — opcode 1/2
│   ├── row_mul       (math_bf16_multiplier)— opcode 3
│   ├── row_mac       (math_bf16_fma)          — opcode 4, with
alu_infer_acc
│   ├── row_minmax    (math_bf16_min/max)      — opcode 5
│   ├── row_compare   (math_bf16_comparator)— opcode 6
│   └── row_relu      (math_bf16_relu)         — opcode 7
├── alu_infer_pipe                             — tag FIFO + arrival join
└── alu_infer_acc                              — fp32 accumulator bank (2
lanes)
```

The as-built RTL folds the planned `alu_infer_decompose`, `alu_infer_select`, `alu_infer_compose`, and `alu_infer_decode` submodules of earlier drafts into `alu_infer_rows.sv`. The four source files are `alu_infer_top.sv`, `alu_infer_rows.sv`, `alu_infer_pipe.sv`, and `alu_infer_acc.sv` in `projects/components/compute-eng-ip/alu-infer/rtl/`.

The tree is flat on purpose: every child is one rank of the block diagram, the data flows strictly top-down through them, and growth rows extend `alu_infer_rows` without touching any sibling — the CGRA replacement boundary of the cell PRD §11 runs around this whole tree.

`alu_infer_pipe` is what makes the elastic handshake (Chapter 3) true — and it is deliberately *not* a bank of padding stages. It is a **tag FIFO with an arrival join** (owner ruling 2026-10-18, as-built since 2026-10-19): each issue takes the next 8-bit tag from a free-running counter, common to every op the invocation loads into the pipeline; the FIFO tracks the tags in flight in issue order; and an entry's result is offered only when every op of its tag group has arrived. v1 issues one row-op per invocation, so a tag group is a single arrival — the join is exercised structurally and multi-op groups (parallel issue of ops that must finish together) are the growth path the mechanism was ruled for. Quick rows (copy, mul, min/max, compare, relu, mac) compute combinationally and land at the issue edge; the add/sub row lands `ADDER_LATENCY` cycles later and re-joins its entry by tag compare — not by position — so a row's pipeline depth is its own business. The join yields group-aligned, strictly in-order retirement at natural per-row latency, and it "naturally takes care of the elastic nature" (owner, ibid.): a stalled consumer simply leaves the head entry offered while rows behind it keep landing, and `ready` falls only when the FIFO is genuinely full — never earlier. Only rows whose *timing* genuinely requires depth carry real stages: the adder as configured (its `PIPE_STAGE_1..4` = `ADDER_LATENCY`), and whatever the integration clock demands of the multiplier/FMA (§7 growth — the join absorbs deeper rows unchanged). Per-row natural latency is deliberately a knob: an aggressively optimized row (a faster multiplier or FMA, say) lands in `rtl/math` and simply joins earlier — the interface never changes (owner direction 2026-10-17: latency work happens on FPGA).

10.2.1 Figure 2.1: ALU-INFER full block diagram

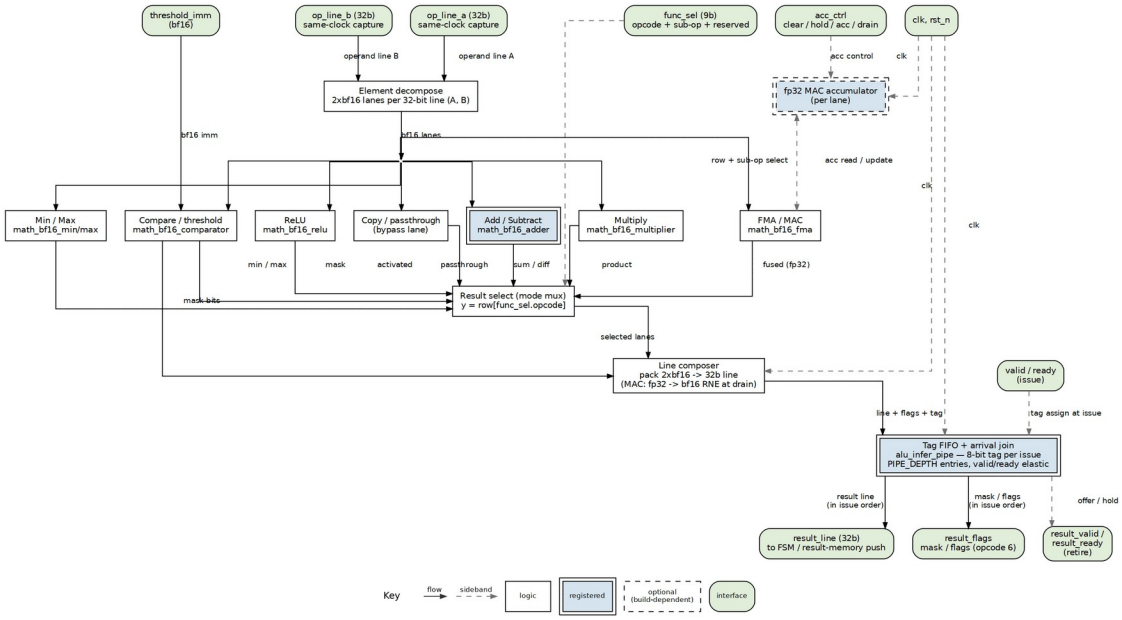


Figure 2.1: ALU-INFER full block diagram

White boxes are ports (the top rank is the interface of Chapter 3); blue boxes are this block's logic — double-bordered where registered (the adder row, the tag FIFO + arrival join); dashed boxes are state (`alu_infer_acc`) and sideband (`clk, rst_n`). Every arrow is labeled with what it carries — the operand trunk splits once at the decompose output and fans to the seven rows, exactly the shared-trunk pattern of the house style.

10.3 Parameters

Table 2.1: `alu_infer_top` parameters

Parameter	Default	Meaning
`LINE_W`	32	Line width in bits; wider lines scale the lane count, never the element width (PRD §2)
`N_LANES`	`LINE_W / 16`	bf16 lanes per line (2 at `LINE_W = 32`)
`ACC_W`	32	fp32 accumulator width per lane
`PIPE_DEPTH`	8	Tag-FIFO entries = maximum invocations in flight; valid range [2,16]. Sizes the stall tolerance, never a latency — presentation is at natural per-row speed
`ADDER_LATENCY`	3	Pipe stages of the add/sub row's `math_bf16_adder` (`PIPE_STAGE_1..4`); valid range [0,4]
`TAG_W`	8	Alignment tag width in bits (owner ruling 2026-10-18); 256 values can never alias the ≤16 entries in flight

No behavior-switch parameters exist in v1: the element model is bf16 only, and the `func_sel` element-width field is reserved (Chapter 6).

10.4 Clock and Reset

Single clock, active-low async `aresetn` (house `ALWAYS_FF_RST`) — matching the cell and the tile's single-clock-domain decision (TILE_SPEC §9). Reset clears the accumulator bank and the registered config snapshot (`func_sel`, `acc_ctrl`); it does not clear anything else, because nothing else is state.

Last Updated: 2026-10-19

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

11 The Seven Function Rows

Each row is one RTL table entry: same lanes in, two lanes out, no state. The `rtl/math` block named per row is the implementation base — its internal structure, pipeline behavior, and special-value handling are documented in its own rtl-math page and are not restated here (one source per fact). What this chapter pins is the row's *wiring* into the ALU: operands, lane routing, and the one or two integration decisions the block does not make.

11.1 Row Summary

Table 2.2: the v1 function rows (normative list in PRD §3)

Opcode	Row	Library block	Operands	Notes
0	copy	wires (bypass)	1 line	passthrough; latency-free
1	add	`math_bf16_adder`	2 lines	per-lane
2	sub	`math_bf16_adder`	2 lines	sign-flip B's sign bit pre-add (see below)
3	mul	`math_bf16_multiplier`	2 lines	mantissa_mult + exponent_adder inside
4	mac	`math_bf16_fma`	2 lines + acc	the workhorse; see `alu_infer_acc` chapter
5	min / max	`math_bf16_min` / `math_bf16_max`	2 lines	sub-op selects; NaN semantics per M1
6	compare / threshold	`math_bf16_comparator`	2 lines or line + imm	mask bits, not values; imm encoding M4

Opcode	Row	Library block	Operands	Notes
7	relu	`math_bf16_relu`	1 line	per-lane; NaN passthrough per M1

11.2 Per-Row Notes

copy (opcode 0). A bypass lane from operand A's lanes to the result select. It exists so the FSM can move lines through the ALU without a function — useful for drain and test patterns. No library block, no latency, no flags.

add / sub (opcodes 1–2). One `math_bf16_adder` serves both rows. The sub row flips operand B's sign bit before the adder — the standard sign-magnitude subtract — so the two rows share one block instance selected by the row decode. Rounding is the library's RNE; the adder's IEEE exception outputs are observable inside the row and are *not* forwarded in v1 (open item M2).

mul (opcode 3). `math_bf16_multiplier` (its `mantissa_mult` and `exponent_adder` children do the field work). "Low half" is meaningless in floating point — the product is the rounded bf16 result, per the PRD's bf16 reframing of the cell PRD's integer "low half" row.

mac (opcode 4). `math_bf16_fma`, consuming the accumulator bank; the row's micro-architecture *is* the next chapter.

min / max (opcode 5). Two library blocks (`math_bf16_min`, `math_bf16_max`) behind the sub-op bit; both compare through `math_bf16_comparator`. The only open question with hardware consequences is NaN propagation (IEEE `minNum` quiet-NaN semantics vs the inference "ignore NaN" convention) — the library blocks' existing semantics decide, and the choice is recorded at M1 so the golden model agrees.

compare / threshold (opcode 6). `math_bf16_comparator` per lane, comparing A against B or against `threshold_imm` (sub-op selects). The row's output is **mask bits, not values** — each lane contributes one bit, packed by the line composer into `result_flags` (format closed: one bit per lane, packed low — M5). `threshold_imm` carries a raw bf16 bit pattern in the simplest encoding (alternative: small int compared after conversion — M4).

relu (opcode 7). `math_bf16_relu` per lane: negative lanes go to +0, positives and the sign of zero pass per the library's contract. NaN passthrough is the library default and is carried at M1 so the golden model states it explicitly.

11.3 What Every Row Shares

- Two lanes computed **in parallel, independently** — lane *i* of a result never depends on lane *j* (PRD §4 invariant 1). This is what lets the testbench check a lane against a scalar golden reference.
- **Natural speed, joined by tag.** As the library ships, every row except add/sub is one combinational cloud (latency 0), and the adder carries `ADDER_LATENCY` of its optional `PIPE_STAGE_1..4`. The ALU does not force fast rows through deep pipe stages to match: each row computes at its natural speed and carries the invocation's 8-bit tag through its own pipeline; `alu_infer_pipe`'s tag FIFO re-joins each result to its entry — by tag compare, not position — and retires strictly in issue order (Chapter 3). A fast row behind a slow one simply waits at the FIFO head (owner ruling 2026-10-18: following ops align with or after the earlier slower operation). A row whose timing genuinely needs depth — a multiplier restructured for the integration clock, the deep-pipelined FMA of §7 — carries real stages and its tag with them; that work belongs to `rtl/math` (the `PIPE_STAGE` family precedent) and does not change this block's contract.
- Growth rows (gelu, sigmoid, silu, tanh, leaky-relu, reciprocal, divide, conversions — PRD §3 growth list) are additional children of `alu_infer_rows` behind free opcodes, added without touching the handshake or the select width.

Last Updated: 2026-10-19

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

12 The FP32 MAC Accumulator

12.1 Why fp32

`alu_infer_acc` is the block's only state, and it exists because of one numerical fact: a bf16×bf16 product is **exact** in fp32 — 8 significand bits times 8 significand bits is 16, well inside fp32's 24 — so a fused multiply-add can accumulate inference layers in fp32 and only round to bf16 when a result line is presented. bf16 accumulate would wrap every operation and drift over a long chain; fp32 accumulate with one RNE rounding at presentation is the production pattern (TPU-class accelerators do exactly this) and is the owner decision of 2026-10-17 recorded in PRD §3.

12.2 Structure

One bank, `N_LANES` (2) fp32 accumulator registers, each private to its lane. `row_mac` feeds the bank and reads it back through `math_bf16_fma`: `acc[i] <= fp32( a[i] × b[i] ) + acc[i]`, with the fused single rounding the FMA block provides. The bank's only control is `acc_ctrl[1:0]` (encoding in Chapter 6):

Table 2.3: accumulator actions

acc_ctrl	Action
HOLD	acc unchanged (default)
CLEAR	acc ← +0.0 in every lane (layer boundary)
ACCUMULATE	acc ← fma(a, b, acc) per lane (a mac invocation)
DRAIN	present acc rounded to bf16; acc unchanged

12.3 The Loop-Carried Dependence (why the row is split)

A dependent MAC chain is a recurrence: step *k* reads the accumulator that step *k−1* wrote. Issue rate does not set the chain's speed — the **feedback loop's latency** does. If the whole FMA were pushed through an 8-stage pipeline, the chain would run at one step per 8 clocks no matter how fast issues arrive. The row is therefore split:

- **Product stage(s)** — the bf16 multiply, whose fp32 product is exact — pipelined as deep as the integration clock wants (§7 growth). Feed-forward, so back-to-back invocations sustain II=1 regardless of depth; the tag FIFO absorbs the depth unchanged.
- **Accumulate stage** — one fp32 add in the feedback path, single-cycle. The chain closes in one clock: one fused accumulate per lane per cycle, sustained.

The product pipeline and the single-cycle accumulate also compose with the tag-join contract: the bf16 view of step *k* retires in issue order at the row's natural latency, like every other row.

12.4 Presentation Semantics (MAS decision M7)

The PRD's interface contract says every v1 row produces one result line per invocation, and this specification decides what a mac invocation presents: **the post-update accumulator, rounded to bf16, as the result line** — a running, observable accumulator view. `DRAIN` then means "present without accumulating," which is how a layer's final values are read out without a following accumulate. Accumulation itself stays in fp32 whatever the presentation; the bf16 view is lossy but the state is not. (This is the MAS reading of PRD §4's "one line per invocation"; flagged M7 in Chapter 6 so the owner can veto.)

12.5 Isolation Invariants

- The bank is readable and writable **only** by `row_mac` (PRD §4 invariant 2) — no other row's datapath crosses it.
- `CLEAR` is the only way to zero it; reset clears it; nothing else disturbs it, so a layer's accumulation chain is reproducible from `{reset, acc_ctrl sequence, operand sequence}` alone.
- Because the bank is per-lane, lane independence (PRD §4 invariant 1) survives mac: lane *i*'s accumulator never sees lane *j*'s operands.

The accumulator chain's cycle-level timing appears in Chapter 4 (Waveform 4.1); verification of long chains and clear/hold corners is Chapter 5.

Last Updated: 2026-10-19

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

13 Result Select and Line Composer

13.1 Result Select (`alu_infer_select`)

A mode mux, not an arbiter: `y = row[func_sel.opcode]`. The decode is a `unique case` on `func_sel.opcode` with exactly the seven row outputs of Table 2.2 as the case arms and no default path to a row — an opcode in the 8–31 growth space selects the copy row's output only in a debug configuration; in the shipped build an out-of-range opcode raises the cell's error IRQ through the status path rather than producing a real result (the cell PRD §8 error hook; the exact signaling is cell-RTL scope). Owner ruling 2026-10-17: the errored invocation **still retires its slot** with a don't-care result line — the elastic pipeline never deadlocks and the FSM's count pairing never breaks. Sub-op bits travel with the row: min/max selection is resolved inside `row_minmax`'s own decode, compare/threshold's operand choice inside `row_compare` — the select mux itself keys on opcode only.

This is the deliberate anti-pest decision of the design: because the select is a port-driven mux, there is no arbitration state, no fairness, no starvation — and the house-style block diagram correctly shows a plain box, not a trapezium.

13.2 Line Composer (`alu_infer_compose`)

Two jobs. For every row except mac: pack the selected row's two bf16 lanes back into one 32-bit line — lane 0 to bits [15:0], lane 1 to bits [31:16] (order fixed at MAS level so testbenches and the golden model agree with the cell PRD's memory layout). For mac: the same packing after the fp32→bf16 RNE rounding that presents the accumulator (Chapter 2.3, decision M7). The composer is also where `result_flags` is assembled for opcode 6: one mask bit per lane, packed into the flags field whose exact format is open item M5.

13.3 Registered Config

`func_sel` and `acc_ctrl` are sampled at the issue edge (`valid && ready`), and the invocation runs on the sampled copy — so the FSM may present the next `func_sel` while the current invocation computes. The registered-config flop is the block's only config state besides the accumulator; reset clears it (Chapter 2.1).

Last Updated: 2026-10-19

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

14 The ALU↔FSM Interface

The seam between the cell's execution FSM and this block. It is the entire external surface of the ALU: the FSM owns sequencing, memories, and the credit loop, and reaches the ALU only through these signals plus clock and reset.

14.1 Signal Contract

Table 3.1: ALU🔄FSM signals

Signal	Dir (ALU view)	Width	Semantics
`op_line_a`	in	`LINE_W`	operand line A (the function's `OP_LINES` count is across both ports: 1 for copy/relu, the A+B pair for binary/mac) — presented together with `op_line_b` and captured at the issue edge; owner ruling 2026-10-17: two ops scheduled in the same clock finish at the same time
`op_line_b`	in	`LINE_W`	operand line B (idle for unary rows)
`threshold_imm`	in	16	bf16 immediate for opcode 6
`func_sel`	in	9	{opcode[4:0], sub_op[6:5], reserved[8:7]} — sampled at issue
`acc_ctrl`	in	2	accumulator action for a mac invocation (Table 2.3)
`valid`	in	1	FSM presents an invocation; operands and config stay stable

Signal	Dir (ALU view)	Width	Semantics
			while `valid` is high and `ready` is low
`ready`	out	1	the ALU can accept an invocation at this edge — high in every cycle that has room (sustained, that is every clock) and low only when a downstream stall backs up into the block
`result_valid`	out	1	a result is offered; stays high with stable data until consumed
`result_ready`	in	1	FSM consumes the offered result
`result_line`	out	`LINE_W`	the offered result line — one per invocation, in issue order
`result_flags`	out	`LINE_W`	opcode-6 mask — one bit per lane, packed low (M5 closed by owner ruling 2026-10-17); zero otherwise
`clk`, `aresetn`	in	1	house single clock, async active-low reset

An **issue** is a rising edge with `valid && ready`; a **retire** is a rising edge with `result_valid && result_ready`. (The `accept` of pre-v0.6 notes is exactly the issue event, not a pin.) `OP_LINES` (cell PRD §7) is the per-function operand-line count — 1 for copy/relu (the A line only), 2 for the binary

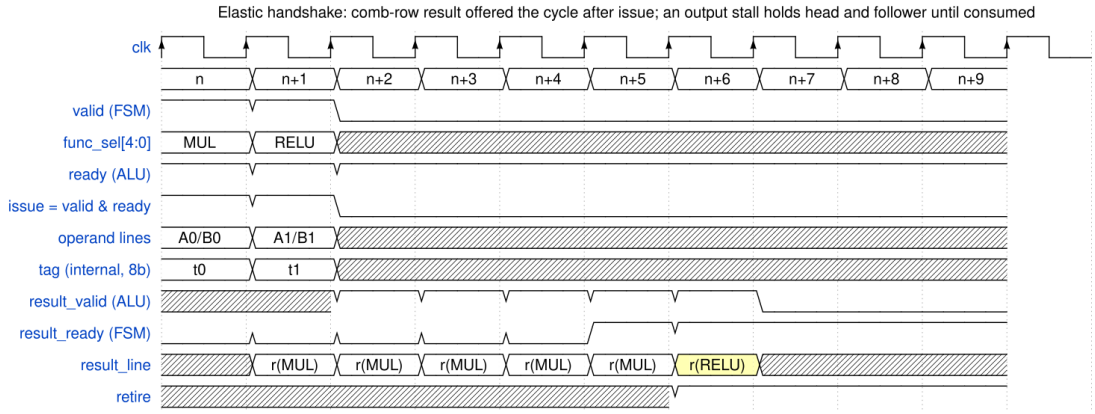
rows and mac (the A line and the B line) — and the two lines arrive **in the same clock** and are captured at the issue edge; the count never appears as a beat sequence on this interface.

14.2 Handshake Rules

1. **Issue is `valid && ready`.** The invocation enters the selected row's pipeline only when both are high at the edge. `ready` is high whenever the block has room, so the FSM may present a new invocation **every clock** (II=1); it deasserts only under downstream backpressure, and while it is low the FSM holds `valid`, the operand lines, `func_sel`, and `acc_ctrl` stable — the standard valid/ready contract.
2. **Retire is `result_valid && result_ready`, strictly in issue order.** Each result is offered at its row's natural latency — combinational rows (copy, mul, min/max, compare, relu, mac) in the cycle after the issue edge, add/sub `ADDER_LATENCY` cycles after — never before every op of its tag group has arrived, and never overtaking an earlier invocation: a fast row behind a slow one waits at the FIFO head. When `result_ready` is low, `result_valid` holds with stable data and the results behind it wait in order — no loss, no reorder, no tags on the seam.
3. **The pipeline is elastic end to end.** Every stage boundary obeys valid/ready, and the tag FIFO is what makes it natural: a stall at the result port leaves the head entry offered with stable data while rows behind it keep landing in their own entries — issue remains open until the FIFO is *genuinely full* (`PIPE_DEPTH` in flight), and only then does `ready` fall. There is no alignment state to freeze: entries hold their arrivals indefinitely. When `result_ready` returns, the head retires and the entries behind it drain in order at one per cycle; issue resumes the moment an entry frees.
4. **In order by construction.** The tag FIFO retires in issue order, so the FSM pairs result N with invocation N by counting. The 8-bit tags are internal to the block (owner ruling 2026-10-18): they never appear on this seam, whose flow control is exactly the two valid/ready pairs. Quick rows compute in parallel at their natural speed and align at the join — `alu_infer_pipe` is a tag FIFO, **not** a bank of padded pipe stages (owner direction 2026-10-17/18).
5. **Config sampled at issue.** `func_sel` and `acc_ctrl` are sampled at the issue edge and travel with the invocation through the pipeline, so the FSM may change them for the next issue immediately.
6. **No error signal on this interface in v1.** An out-of-range opcode is a cell-level error (Chapter 2.4); the library blocks' IEEE exception outputs are not forwarded (open item M2).

The one place uniform latency is *engineered* rather than padded is opcode 4: a dependent MAC chain feeds back through the accumulator, so its sustained rate is set by the feedback loop's latency, not the issue rate. The mac row is therefore split — pipelined exact-fp32-product stage(s) ahead, single-cycle fp32 accumulate stage in the feedback path — so a chain sustains II=1 with the output draining (Chapter 2.3). **Scheduler-paced feedback** (owner ruling 2026-10-17): whenever results feed back — opcode 4's chain today, any future recurrence — the scheduler owns the pace entirely; it shapes the `valid` stream and obeys `ready`, and the ALU makes no autonomous pacing decisions. The single-cycle accumulate exists so that scheduler-chosen II=1 is sustainable.

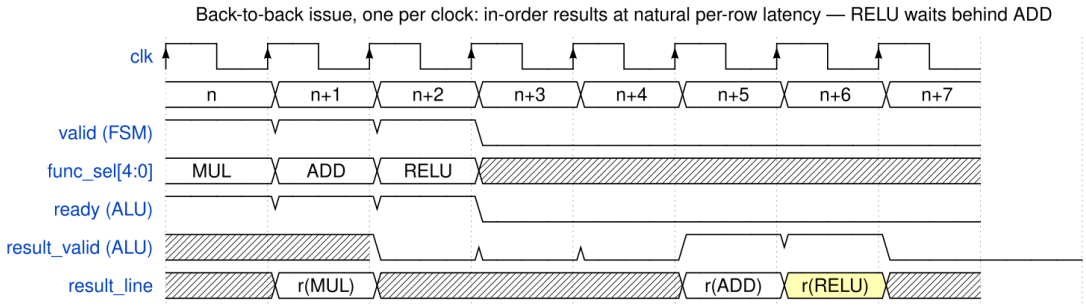
14.2.1 Waveform 3.1: Elastic handshake — natural latency, output stall, ready stays high



Waveform 3.1: elastic handshake

Two invocations ride the elastic protocol end to end. Both are combinational rows: the MUL issued at n is offered one cycle later (n+1), the RELU issued at n+1 lands at n+2 — each taking its 8-bit tag (t0, t1) at issue. The consumer stalls from n+2 to n+4: the head holds `r(MUL)` with stable data, the RELU waits behind it in its own FIFO entry, and **`ready` stays high throughout** — with `PIPE_DEPTH` entries, a two-entry backup is nowhere near full, so the input side never feels the stall. The consumer accepts at n+5 (MUL retires) and n+6 (the waiting RELU retires immediately after). Operands and config are stable from `valid` to issue; both operand lines are captured at the issue edge.

14.2.2 Waveform 3.2: Back-to-back issue — mixed latencies, in-order results



Waveform 3.2: back-to-back issue

Three invocations (MUL, ADD, RELU) issue on consecutive clocks with the output draining. The MUL (combinational) retires at n+1. The head then belongs to the ADD, whose pipelined row presents at n+5 (`ADDER_LATENCY = 3`); `result_valid` dips while the faster results behind it — the RELU, computed at n+3 — wait at the join. ADD retires at n+5, RELU immediately after. This is the owner ruling of 2026-10-18 in one picture: following ops are delayed to align with or after the earlier slower operation, and no row carries padded stages to hide the difference.

14.3 Boundary Notes

- **Config plane.** The cell PRD's §7 register map (`FUNC_SEL`, `OP_LINES`, ...) is AXIL and cell-RTL scope; this block sees config only as the registered `func_sel`/`acc_ctrl` inputs above. The in-band `cfg` packets that write those registers at layer boundaries travel with the delta traffic and are demuxed by the cell, never by this block.
- **Tetrad control core.** The kestrel-class per-tetrad core reads cell status and IRQs and exercises operational registers over the AXIL plane (alu-infer PRD §5; TILE_SPEC §11.13); it has no direct signals here by construction.

Last Updated: 2026-10-17

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

15 The Compute Cycle, per Opcode Class

Every invocation is the same three beats — **pop, compute, push** — but under the elastic handshake (Chapter 3) the beats of consecutive invocations overlap in the pipeline: while one invocation computes, the next pops and the previous pushes. This chapter walks each opcode class once as if it were alone; the overlap rules at the end apply to all of them together.

15.1 Unary Rows (copy, relu)

The FSM presents its single line on `op_line_a` (operand B idle). The row computes both lanes in parallel; the composer packs the result. Copy adds nothing; relu clamps negative lanes to +0 per the library block. One pop, one push, no state touched.

15.2 Binary Rows (add, sub, mul, min/max)

Both operand lines — one of A, one of B — arrive **in the same clock**; the owner ruling of 2026-10-17 is that two ops scheduled in the same clock finish at the same time. The decompose block splits the two lines into lanes at the issue edge, so the row sees lane pairs (A[0],B[0]) and (A[1],B[1]) for the same element positions, captured together. The row computes both lane pairs independently (PRD §4 invariant 1); the select forwards; the composer packs. Sub shares the adder with sign-flipped B (Chapter 2.2). Min/max resolve the sub-op internally (Chapter 2.4).

15.3 Compare / Threshold

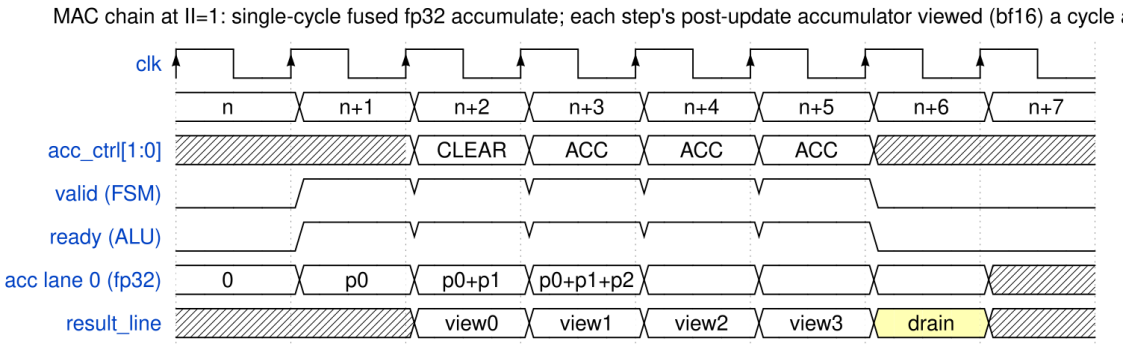
Like a binary row, except the comparator's per-lane decision bits — A < B per lane, or A < imm when the sub-op selects the immediate form — are what get packed, into `result_flags` (format M5), while `result_line` carries the mask bits replicated or zero per the same open item. `threshold_imm` must be stable through issue like any operand.

15.4 MAC (the chain that matters)

A mac invocation is `acc[i] ← fma(a[i], b[i], acc[i])` per lane, with `acc_ctrl` choosing the wrapper behavior:

1. **CLEAR** at the layer boundary: `acc` ← +0.0, applied in issue order at the accumulate stage so a following ACC never sees stale state. The invocation **flows through the pipeline** like any other (owner ruling 2026-10-17): no operands are consumed, and its presentation slot retires a don't-care result line — every issued invocation retires, so the FSM's count pairing never breaks.
2. **ACCUMULATE** per step of the chain: pop the A and B lines — presented together in the issue clock — and fused-accumulate. The result line presents the post-update accumulator rounded to bf16 (decision M7) — observable, lossy, and cheap. Because the row is split (Chapter 2.3) — pipelined exact product ahead, one-cycle fp32 accumulate in the feedback path — the chain can sustain one step per clock when the scheduler issues that fast (feedback pacing is the scheduler's — Chapter 3); the per-step bf16 views on the result line lag the accumulator state by the product-pipe depth and retire in order with everything else.
3. **DRAIN** for the layer's final values: present the rounded accumulator without accumulating. After drain the FSM re-points the result memory or starts the next layer with CLEAR.

15.4.1 Waveform 4.1: MAC chain — clear, three accumulates, drain



Waveform 4.1: MAC chain

The numerics that make this safe: each product is exact in fp32 before the add, so the only rounding before presentation is the FMA's single rounding per step, and the bf16 views on the result line never feed back into the accumulator — the state stays fp32 end to end. A long chain is

therefore reproducible from `{reset, acc_ctrl sequence, operand sequence}`, which is exactly the property Chapter 5's accumulator tests rely on.

15.5 Overlapping Invocations

The pipeline never inserts a stall of its own: a MUL issue on one clock, a RELU on the next, a MAC step on the third — all in flight together, results retiring in issue order as each tag group arrives (Waveform 3.2). The one stall source is downstream: when `result_ready` deasserts, the head entry holds its offer with stable data, rows behind it keep landing in their own entries, and `ready` falls only once the FIFO is full (Waveform 3.1). Three consequences are worth stating:

- **No drain logic.** Because retirement is in issue order regardless of per-row latency, switching opcode mid-stream costs nothing; there is no pipeline flush, no bubble. Backpressure needs no drain either — the FIFO entries simply hold their state until the blockage clears.
- **The FSM pairs results by counting.** Result N belongs to invocation N; no tags ride the interface.
- **Idle is a hold.** When the FSM stops issuing (`valid` low), the protocol needs nothing further: in-flight invocations drain, the last result holds until consumed, and the accumulator bank holds until `acc_ctrl` says otherwise.

The alignment mechanism (owner ruling 2026-10-18, as-built since 2026-10-19). When an invocation is loaded it takes an **8-bit tag**, common to all ops that must execute in parallel; a **FIFO** tracks the tags in flight; an op asserts its valid only when every op of its tag group is ready. The as-built `alu_infer_pipe` is exactly this: a free-running tag counter at issue, a `PIPE_DEPTH`-entry FIFO in issue order, and an arrival join — combinational rows land at the issue edge, the pipelined add/sub row re-joins its entry by **tag compare** (not position) when its result lands. v1 issues one row-op per invocation, so a tag group is one arrival; the group join is exercised structurally and multi-op parallel issue is the growth path the ruling targets (deep-pipelined rows, PRD §7). This supersedes the first v1 build's alignment-timer mailbox and its uniform `ROW_LATENCY` presentation — with no interface change: the tags stay inside the block; the ALU↔FSM seam keeps its two valid/ready pairs.

Last Updated: 2026-10-19

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

16 Verification Hooks and Observability

The ALU's verifiability is a consequence of its structure, and this chapter names the hooks the design exposes on purpose. The tests live in `alu-infer/dv/` — `tbclasses/alu_infer_tb.py` holds the testbench (an `AluInferTB` class on the repo's `TBBase`), and `tests/test_alu_infer.py` is a thin TEST_TYPE-dispatching runner, the repository standard pattern (the stream area's). Both valid/ready interfaces are owned by RDS-DV GAXI BFMs: a `GAXIMaster` drives the issue interface (`valid`/`ready` + operand/func fields), a `GAXISlave` receives the result interface (`result_valid`/`result_ready` + line/flags), and passive `GAXIMonitors` record completed transfers on both sides. Timing checks anchor on the BFMs' packet timestamps (the master stamps the accepting edge, the slave the retiring cycle), and the mixed soak additionally runs under the standard randomized GAXI delay profiles (`GAXI_TIMING_PROFILE`, from the shared `GAXI_RANDOMIZER_CONFIGS` table in `bin/TBClasses/amba/amba_random_configs.py`): func sweeps `fast`, `slow_producer`, `slow_consumer`, `gaxi_realistic`, `gaxi_backpressure`; full adds `constrained`, `burst_pause`, `high_throughput`, `gaxi_stress` (the per-type default is `backtoback`). The scenario-to-coverage map lives in `dv/testplans/alu_infer_top_testplan.yaml` — its `covers_lines` are generated from the per-scenario merged coverage data, and `bin/cov_utils/verify_testplan_coverage.py --strict` re-checks every scenario against a fresh `COVERAGE=1` run. The cell PRD's credit-loop tests already prove lines-in → credits-out independently of the datapath, so these hooks assume a datapath-only test space.

16.1 The Golden Model

One scalar bf16 numerics model, applied per lane: compute the expected result in fp32, round to bf16 RNE. This is the same discipline the math library's own dv uses, which means the ALU's golden model and the library's golden model are the same arithmetic — and lane independence (PRD §4 invariant 1) makes a per-lane scalar comparison a *strong* check, not a sampling one. For mac, the golden model keeps a fp32 accumulator per lane and only rounds at presentation, matching decision M7.

16.2 Built-in Cross-check

Every row instantiates an `rtl/math` block with its own passing dv. The unit testbench therefore double-checks for free: drive the same operands to the ALU and to the library block directly (or to its reference model); any divergence is a bug in exactly one of them. This is the strongest hook in the design — it turns “the ALU works” into “the ALU is wired the way the library block expects,” which is where integration bugs actually live.

16.3 Coverage Matrix

Table 5.1: unit coverage matrix

Axis	Values
Opcode	0–7 (all v1 rows)
Special-value class	NaN payload, +inf, −inf, +0, −0, normal, denormal/FTZ (per block `SUBNORMAL_SUPPORT`, M3)
Lane position	0 and 1 (both populated, either populated)
Accumulator	long chains (>100 ACC steps), CLEAR/ACC/DRAIN corners, clear-then-accumulate, drain-without-accumulate

Special values are mandatory corners, not extras — the bf16_testing base classes (`bf16_special_value_product`, per the math area's TASK-006 grid discipline) generate the Cartesian product; the matrix above says where it must be applied.

16.4 Elastic and Tag-Join Behavior

The alignment mechanism has its own directed tests — the as-built TEST_TYPEs `copy_timing`, `backpressure`, and `tag_join`: natural per-row latency (combinational rows offered the cycle after issue, add/sub `ADDER_LATENCY` cycles after, measured from BFM packet timestamps), II=1 streaming proven from consecutive retirement deltas under `send_burst`, the elastic hold-and-resume (the slave's `set_ready_policy('stall'/'always')` drives the consumer side), the tag-FIFO fill (`ready` falls only at `PIPE_DEPTH` in flight, never earlier), and the join itself — an add issued ahead of a relu and a copy must present first, with the fast rows waiting at the FIFO head. These are the tests that pin the owner ruling of 2026-10-18.

16.5 Observability

- **No internal monitor taps by default.** Observability is the Ocelus MonBus plane's job at tile scope; the ALU contributes nothing to it.
- **The accumulator bank is read-back-able** through normal mac DRAIN invocations, so no debug port is needed for the state that matters.
- **Open item M2** (flag observability) is the one place this chapter deliberately leaves a hook decision open: whether the library blocks' IEEE exception outputs forward into `result_flags`, and if so in what layout alongside the mask bits (the mask format itself is closed — one bit per lane, packed low; M5).

16.6 Test Levels

House gate/func/full grid, per the cell PRD §9: gate smoke on copy + add; func per-row truth tables against the golden model; full as the coverage matrix above plus the cell's integration tests (in-band `cfg` programming, credit loop, packetizer) that consume this block.

Last Updated: 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

17 Parameters, Encodings, and Open Items

17.1 Parameters

See Table 2.1: `LINE_W = 32`, `N_LANES = LINE_W/16`, `ACC_W = 32`, `PIPE_DEPTH` (tag-FIFO entries, default 8 — the stall-tolerance knob), `ADDER_LATENCY` (the add/sub row's own pipe depth, default 3), and `TAG_W` (8, per the owner ruling). The only behavior-switch surface is the opcode table itself — the element model is bf16-only and rounding is the library's RNE throughout (a dynamic rounding mode is out of scope by the garuda RNE precedent; any request would raise the error path, per Chapter 2.4).

17.2 `func_sel` Encoding

Table 6.1: `func_sel` fields

Bits	Field	Encoding
[4:0]	opcode	0 copy, 1 add, 2 sub, 3 mul, 4 mac, 5 min/max, 6 compare/threshold, 7 relu; 8–31 growth
[6:5]	sub_op	row-internal (min/max select; compare operand select); row docs of Chapter 2
[8:7]	reserved	element-width legacy field — reserved under the bf16 model (M6)

17.3 `acc_ctrl` Encoding

Table 6.2: `acc_ctrl` encoding (accepted 2026-10-17 — "works for now"; revisit at FPGA bring-up if the mac row is reworked)

Value	Name	Action
2'b00	HOLD	acc unchanged
2'b01	ACCUMULATE	per-lane `acc <= fma(a, b, acc)`
2'b10	CLEAR	acc ← +0.0 per lane
2'b11	DRAIN	present acc (fp32→bf16 RNE) without accumulating

17.4 Filelists

The RTL lives in the alu-infer component (`alu-infer/rtl/...`, organized like the other component areas — flat RTL, per-block filelists, and a `<area>_all.f` master for lint). The block has its own unit filelist, `alu-infer/rtl/filelists/alu_infer_top.f` — the four ALU sources plus the `rtl/math` filelists of the instantiated library blocks — which is what the unit testbench (`alu-infer/dv/tests/test_alu_infer.py`) compiles.
`alu-infer/rtl/filelists/alu_infer_all.f` is the lint master. The cell-level filelists (`cell_top.f`, `cell_all.f`) include the block's filelist for integration, since slice 3+ instantiates `alu_infer_top` inside the cell.

17.5 Open Items Register

Carried from PRD §8, with the MAS additions M7; these are the only decisions this specification does not make.

Table 6.3: open items M1–M8

ID	Item	Blocking
M1	NaN/inf semantics in min/max and activations (IEEE `minNum` propagation vs inference “ignore NaN”)	golden model for rows 5, 7
M2	Whether library IEEE exception outputs forward into `result_flags`	observability only
M3	Denormal/FTZ policy cell-wide vs per-block `SUBNORMAL_SUPPORT` defaults	special-value grid
M4	`threshold_imm` encoding (raw bf16 pattern vs converted small int)	opcode-6 tests
M5 — **closed 2026-10-17 (owner ruling)**	`result_flags` format: one mask bit per lane, packed low in a `LINE_W` line; port width `LINE_W`	closed
M6	`FUNC_SEL` reserved-field disposition (retire vs keep for fp8/integer growth)	register map freeze
M7 — **accepted 2026-10-17**	mac presents the post-update fp32 accumulator rounded	accepted

ID	Item	Blocking
	to bf16 per invocation; `DRAIN` presents without accumulating — accepted as drafted ("works for now"); revisit if FPGA latency work rewrites the mac row	
M8	Per-row natural-latency table and `PIPE_DEPTH`/`ADDER_LATENCY` values — **deferred to FPGA bring-up** (owner direction 2026-10-17): tuned against hardware, with possible aggressive per-function latency minimization; the tag-join contract absorbs faster or deeper rows without interface change (each row carries its tag through its own pipeline and joins by compare)	FPGA bring-up

M1, M3 and M4 resolve from the `rtl/math` library blocks' existing semantics wherever those semantics are already fixed — the work is recording the choice, not inventing it. M2 and M6 are cell-level and ride the cell PRD's next edit. **M5 is closed** (owner ruling 2026-10-17): one mask bit per lane, packed low in a `LINE_W` line. M7 is this MAS's one novel decision and is the first candidate for owner review of this document.

Last Updated: 2026-10-17